

# Winformer: Transcending Pairwise Similarity for Time-series Generation

Haoyi Zhou<sup>1</sup> Xin Xue<sup>1</sup> Tianyu Chen<sup>1</sup> Lanhao Li<sup>1</sup> Lijun Sun<sup>1</sup> Jianxin Li<sup>1</sup>

## Abstract

The periodicity misalignment remains a challenge problem in generating time-series data across multiple domains. The fundamental processing unit of attention in time-series modeling has long been restricted to either individual points or fragmented segments, limiting their ability to capture and adapt to complex periodic patterns inherent in diverse domains. To address this, we introduce Winformer, first to extend this processing unit from individual points to sliding windows, establishing a unified window-wise attention paradigm. Leveraging the adaptive window-alignment kernels derived from the frequency decomposition, Winformer brings semantically richer window representations, and effectively captures and transfers complex periodic patterns across domains. Extensive experiments on 12 real-world datasets demonstrate Winformer’s effectiveness, achieving an average performance gain of 10.67% over SOTA baselines.

## 1. Introduction

When generating data across multiple time-series domains, one of the major issue is the *domain confusion* (Liu et al., 2024) that different domains exhibits diverse temporal distribution. The normalization techniques (Kim et al., 2022; Ulyanov et al., 2016; Ogasawara et al., 2010) could alleviate the magnitude misalignment, and we focus on the phase misalignment (Zhang et al., 2025; Yuan & Qiao, 2024), i.e. periodicity, which may arise from sampling rates, system delays, manual operations, or inconsistencies environment.

Existing approaches models the periodicity in point-wise or patch-wise, both of which fall short in addressing domain confusion. Following the vanilla Transformer design (Vaswani et al., 2017; Zhou et al., 2021), the previous methods (Yuan & Qiao, 2024; Piao et al., 2024; Zhou et al.,

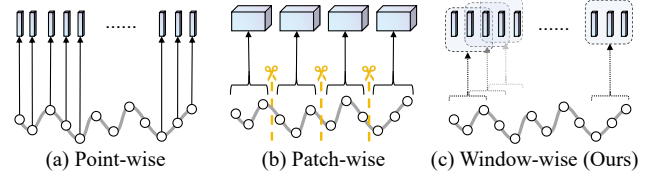


Figure 1. **The perspective of time-series modeling.** (a) Point-to-point modeling on exact interactions (**Slicing** window, window size=1). (b) Patch-to-patch modeling on middle-level interactions (**Slicing** window, window size=3). (c) We propose window-to-window modeling on complex interactions (**Sliding** window, window size=3).

2022) model the exact interactions between individual time-series through a point-wise perspective, as illustrated in Fig. 1(a). With limited representation ability, these methods require additional frequency components to capture long-range and high-order interactions (Zhou et al., 2022) and stacking of attention layers (Wu et al., 2021). Recently, the time-series patching (Nie et al., 2023; Peebles & Xie, 2023) is introduced to obtain middle-level interactions. However, the equidistant patching directly breaks evolving trends and periodic patterns, as in Fig. 1(b). Thus, how to adaptively capture the periodicity across diverse domains remains a challenge in cross-domain time-series generation.

To tackle this critical limitation, we are the first to extend the fundamental principle of attention mechanism, shifting from pairwise point similarity to continuous window comparison. To address this issue, we extend the fundamental principle of attention mechanism, shifting the pairwise point similarity, to windows comparison. As shown in Fig. 1, in point-wise modeling, the slicing window is set as 1 as the attention calculates on pairwise points, while in patch-wise modeling, the slicing window is set as 3 and the attention is forced on pairwise lattices. Unlike the fixed slicing windows used in traditional data processing, our approach provides a new way by introducing the sliding window comparison during attention calculation shown in Fig. 1(c), enabling adaptive periodicity alignment of across domains. Specifically, we propose the Ample attention, which adaptively captures the periodicity with window-wise aligning kernel based on frequency decomposition, enabling the window-wise similarity comparison. With the periodicity aligning across domains, our method effectively enhances domain adaptability thereby improves the generation quality.

<sup>1</sup>Beihang University, Beijing, China. Correspondence to: Jianxin Li <lijx@buaa.edu.cn>.

To summarize, Our contributions are:

- We are the first to rethink the fundamental processing unit of attention for time series and extend it from pairwise point similarity to sliding window comparison. We establish a unified window-wise attention paradigm, providing a principled foundation for periodicity-aware cross-domain generation.
- We reveal a fundamental equivalence between window-wise similarity in the spectral domain and learnable convolutions on the original attention score map, bridging frequency decomposition with convolutional attention. This theoretical result leads to the Ample attention, an efficient mechanism that captures periodic alignment without explicit Fourier transforms.
- Extensive experiments on 12 real-world datasets across four domains demonstrate the effectiveness of the proposed paradigm. Winformer achieves state-of-the-art performance with an average improvement of 10.67% in MMD, and visualization analysis confirms that the window-wise mechanism recovers finer periodic structures during the denoising process.

## 2. Related work

Existing time-series generation models mainly consist of GAN-based models, VAE-based models and diffusion-based models. GAN-based models (Yoon et al., 2019; Jeon et al., 2022) possess advantages of not requiring distribution assumptions, but show poor training stability. VAE-based models (Desai et al., 2021; Lee et al., 2023) can achieve training stability with clear optimization target, but show limited generation quality and diversity. Diffusion models (Huang et al., 2025; Ge et al., 2025), which are popular paradigms, have advantages in both training stability and generation quality.

Transformer-based diffusion models show a more powerful effect in various time-series tasks, including forecasting (Feng et al., 2024), anomaly detection (Wang & Li, 2025) imputation (Alcaraz & Strodthoff, 2023; Tashiro et al., 2021), and generation (Peebles & Xie, 2023; Yuan & Qiao, 2024; Cao et al., 2025). In addition to domain-specific modeling, there are also transformer-based diffusion models for cross-domain generation (Huang et al., 2025; Ge et al., 2025; Ma et al., 2025). However, these methods lack of an alignment mechanism for domain-wise correlations, which require further exploration.

## 3. Preliminary

With the rolling forecasting setting with a fixed horizon, we define the  $t$ -th sequence inputs of domain  $j$  as  $\mathcal{X}^{(j,t)} =$

$\{\mathbf{x}_1^{(j,t)}, \dots, \mathbf{x}_{L_x}^{(j,t)} \mid \mathbf{x}_i^{(j,t)} \in \mathbb{R}^D\}$ , where  $L_x$  stands for the horizon length. We mix the  $M$  datasets from different domains together to get the unified dataset  $\mathcal{X} = \cup_{i=1}^M \mathcal{X}_i$  considering the cross-domain time-series generation settings, and the overall target is to learn a parameterized model  $\theta$  with  $p_\theta(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T | i)$  for the  $i$ -th dataset.

### 3.1. Vanilla Self-attention

The self-attention mechanism (Vaswani et al., 2017) yields successful pairwise alignment ability in sequence modeling. It is calculated on three transformed inputs from  $\mathbf{X} \in \mathbb{R}^{L_x \times D}$ , i.e, query, key and value, which is defined as the scaled dot-product as:

$$A(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V}, \quad (1)$$

where we have  $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q^\top$ ,  $\mathbf{K} = \mathbf{X}\mathbf{W}_K^\top$ ,  $\mathbf{V} = \mathbf{X}\mathbf{W}_V^\top$  with trivial projections  $\mathbf{Q} \in \mathbb{R}^{L_Q \times d}$ ,  $\mathbf{K} \in \mathbb{R}^{L_K \times d}$ ,  $\mathbf{V} \in \mathbb{R}^{L_V \times d}$  and the efficient scaled norm of the dimension  $d$ .

### 3.2. Denoising Diffusion Probabilistic Models

The diffusion probabilistic generation methods are based on the assumption of Markov chain. Specifically, let the input matrix  $\mathbf{X}_{(0)} \in \mathbb{R}^{L_x \times D} \sim q(\mathbf{X})$  be the real data. At the  $n$  step of the diffusion process, we obtain  $\mathbf{X}_{(n)}$  from  $\mathbf{X}_{(n-1)}$  by adding Gaussian noise with the transition kernel defined as  $q(\mathbf{X}_{(n)} | \mathbf{X}_{(n-1)}) = \mathcal{N}(\mathbf{X}_{(n)}; \sqrt{1 - \beta_n} \mathbf{X}_{(n-1)}, \beta_n \mathbf{I})$ , where  $\beta_n \in (0, 1)$ . By recurring the  $n$  steps of the Markov chain, we can derive:

$$q(\mathbf{X}_{(n)} | \mathbf{X}_{(0)}) = \mathcal{N}(\mathbf{X}_{(n)}; \sqrt{\bar{\alpha}_n} \mathbf{X}_{(0)}, (1 - \bar{\alpha}_n) \mathbf{I}), \quad (2)$$

where  $\alpha_n = 1 - \beta_n$  and  $\bar{\alpha}_n = \prod_{i=1}^n \alpha_i$ . In reversing, a learning-based model reconstructs as

$$p_\theta(\mathbf{X}_{(n-1)} | \mathbf{X}_{(n)}) = \mathcal{N}(\mathbf{X}_{(n-1)}; \mu_\theta(\mathbf{X}_{(n)}, n), \Sigma_\theta), \quad (3)$$

where  $\mu_\theta(\mathbf{X}_{(n)}, n)$  is estimated as

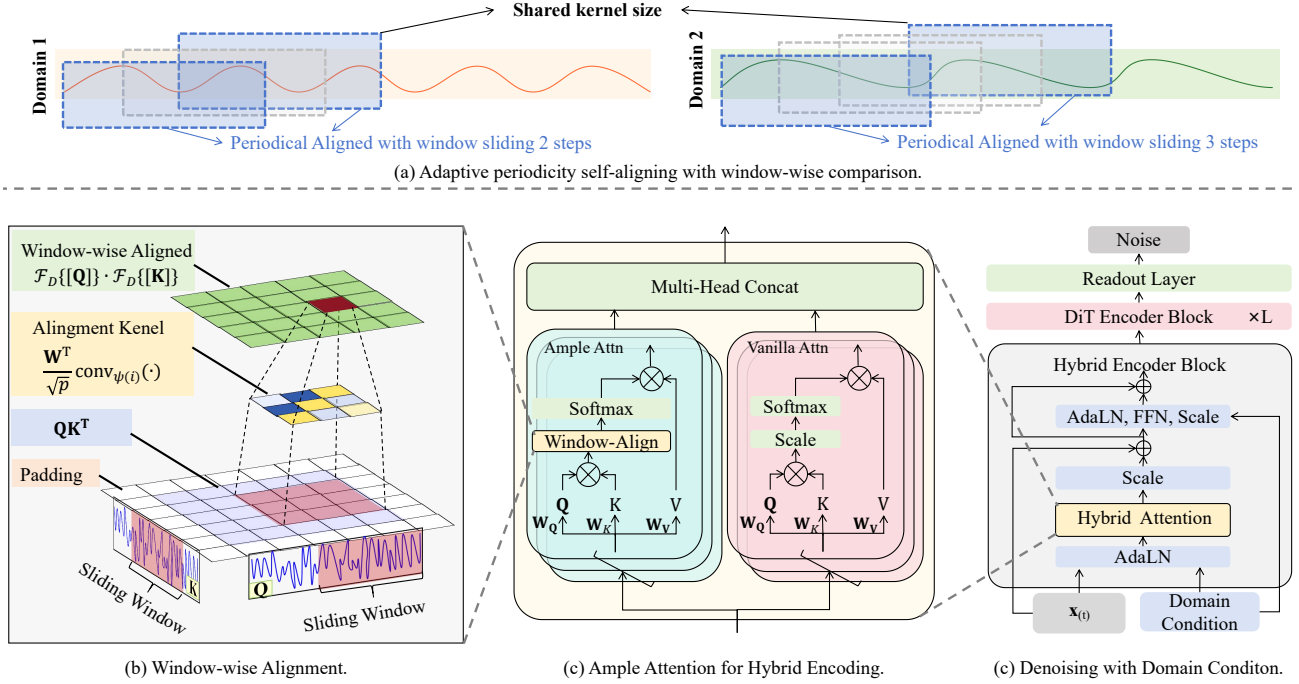
$$\mu_\theta(\mathbf{X}_{(n)}, n) = \frac{1}{\sqrt{\alpha_n}} \left( \mathbf{X}_{(n)} - \frac{\beta_n}{\sqrt{1 - \bar{\alpha}_n}} \epsilon_\theta(\mathbf{X}_{(n)}, n) \right). \quad (4)$$

## 4. Methods

In this section, we introduce the Winformer, a diffusion model with window-wise alignment as illustrated in Fig. 2.

### 4.1. Theoretical Analysis for Window-wise Alignment

The vanilla self-attention mechanism is built upon the point-to-point dot-product similarity. It achieves significant alignment ability over the tokenized inputs of language (Devlin et al., 2019), where the text embedding Word2Vec (Mikolov



**Figure 2. The overall architecture.** (a) **Periodicity Aligning with sliding windows:** Adaptive window-wise alignment allows periodicity self-aligning across diverse domains by sliding kernel with shared kernel size. (b) **Window-wise Alignment:** Based on the frequency decomposition, the window-wise alignment is realized by a convolutional kernel sliding on similarity maps. (c) **Ample Attention for Hybrid Encoding:** The hybrid attention concatenate the heads of Ample attention and vanilla attention for feature fusing. (d) **Denoising with Domain Condition:** With domain conditioning, the denoising network predicts noise for cross-domain time-series generation.

et al., 2013) and Jina (Günther et al., 2023) ensure the unified linguistic space. For time-series generation, the Fourier transform decomposes the sequences into constituent frequencies and defines similarity between the frequency coefficients to make it possible to align the series. Building on this, we propose transforming time-series data into the Fourier basis and calculating similarity over complex planes.

#### 4.1.1. FREQUENCY DECOMPOSITION OVER SIMILARITY

Recalling that we have an input matrix  $\mathbf{X}$ , the  $i$ -th time step is represented by a vector  $\mathbf{x}_i \in \mathbb{R}^{1 \times D}$ . Assume that we perform a window-wise Fourier transformation, over these finite-length signals, and the window size is set as  $p$ . Since the time-series become discrete-time inputs, we use the discrete Fourier transform (DFT)  $\mathcal{F}_D$  instead. We apply the DFT operator to each feature dimension independently, which follows the same setting (Alaa et al., 2021). We select the  $q$ -th dimension of projected inputs  $\mathbf{Q}$  as  $\mathbf{Q}^q = \{\mathbf{x}_1^q, \dots, \mathbf{x}_{L_x}^q\}$ . Through the temporal zero padding on the beginning and ending, we have the window-wise attention inputs  $\mathbf{Q}^q$  and  $\mathbf{K}^q$  respectively:

$$[\mathbf{Q}]_i^{(q)} = \{\mathbf{Q}_j^q | i \times s \leq j < (i+1) \times s\} \quad , \quad (5)$$

and

$$[\mathbf{K}]_i^{(q)} = \{\mathbf{K}_j^q | i \times s \leq j < (i+1) \times s\} \quad , \quad (6)$$

where the  $[\mathbf{Q}]_i^{(q)} \in \mathbb{R}^{s \times 1}$ ,  $[\mathbf{K}]_i^{(q)} \in \mathbb{R}^{s \times 1}$  and  $s$  stands for the stride. For the  $t$ -th time step, the DFT transforms the real-valued inputs into the complex-valued ones as  $\mathcal{F}_D\{[\mathbf{Q}]_t^{(q)}\}$  and  $\mathcal{F}_D\{[\mathbf{K}]_t^{(q)}\}$ . Likewise the self-attention mechanism, we can perform the dot-product between the selected time step  $t_1$  and  $t_2$  in the frequency domain, whose scores calculate the similarity between different spectral components (Zhou et al., 2022; Piao et al., 2024; Kong et al., 2023) as:

$$\tilde{\mathbf{F}}_{(t_1, t_2)}^{(p, q)} = \mathcal{F}_D\{[\mathbf{Q}]_{t_1}^{(q)}\} \cdot \mathcal{F}_D\{[\mathbf{K}]_{t_2}^{(q)}\} \quad . \quad (7)$$

This score is a complex matrix containing the real component  $\text{Re}(\tilde{\mathbf{F}})$  and imaginary component  $\text{Im}(\tilde{\mathbf{F}})$ , a two-channel matrix with shape  $2 \times s \times 1$ . The former component measures the magnitude difference between the spectral coefficients, while the latter one is about the phase difference.

Then, we can concatenate all the inputs' DFT result as  $\tilde{\mathbf{F}}_{(t_1, t_2)}^{(p)} = [\tilde{\mathbf{F}}_{(t_1, t_2)}^{(p, 1)}, \dots, \tilde{\mathbf{F}}_{(t_1, t_2)}^{(p, d)}]$  with a shape of  $2 \times s \times d$ . Likewise the self-attention mechanism, we take the magnitude difference of the spectral coefficients at  $t_1$  and  $t_2$  as an example, which could be normalized by  $\text{Softmax}(\cdot)$  to reorganize  $\mathbf{V}$ . Then we can define the real-valued similarity within the window-to-window comparison as:

$$\mathbf{S}_{(t_1, t_2)}^{\text{Re}} = \text{Re}(\tilde{\mathbf{F}}_{(t_1, t_2)}^{(p)}) \mathbf{1} \quad , \quad (8)$$

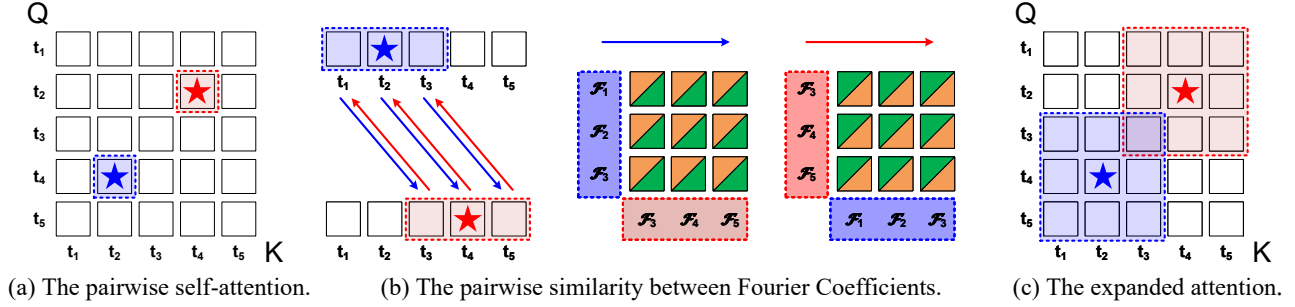


Figure 3. The process of attention expansion. (a) The visualization of pairwise feature map (dot-product) of vanilla self-attention. There are five tokens ( $t_1, t_2, t_3, t_4, t_5$ ) in both query and keys. And the blue star ( $Q_{t_4}, K_{t_2}$ ) and red star ( $Q_{t_2}, K_{t_4}$ ) form a duality. (b) The new similarity is defined between the windowed Fourier coefficients. We set the window size as 3, and the matrix is the dot-product of corresponding coefficients, where the orange triangle stands for real parts and the green triangle for imaginary parts. The blue arrow denotes the grouped similarity from  $(t_1, t_2, t_3)$  to  $(t_3, t_4, t_5)$  like the blue star in figure a. (c) We design the expanded attention to crop the scores from original feature map.

where  $\mathbf{1} \in \mathbb{R}^{d \times 1}$  denotes the all-one vector and it sums the real coefficient of all channels (also applies to imaginary).

As illustrated in Fig. 3, the original self-attention leverages the pairwise similarity to measure the relationship between different inputs, e.g., the blue star represents the attention feature map for  $Q_{t_4}$  and  $K_{t_2}$  pairs and red star is the dual score. If we perform the windowed DFT and measure the “group” similarity between  $(t_1, t_2, t_3)$  and  $(t_3, t_4, t_5)$  through the real-valued similarity defined in Eq.(8) and its imaginary ones, we can acquire the blue arrows composed with real parts (the orange triangle) and imaginary parts (the green triangle). Based on the following derivation, we will demonstrate that the pairwise attention could be expanded to crop scores, where the score (blue star) stands for window-to-window comparison.

#### 4.1.2. THE CONVOLUTIONAL CALCULATION

The Fourier Transform, a mathematical operator (Duhamel & Vetterli, 1990), can be calculated in Eq.(7), but it requires two transformations, namely product in frequency domain and composition of real part and imaginary part. Actually, our ultimate goal is to calculate a new similarity score in window-wise perspective that is numerically compatible with other network components. Inspired by the convolution theorem (McGilllem & Cooper, 1991), which bridges the connection between Fourier operator and convolution operator, we decompose the calculation of the DFT operator as a linear transformation:

$$\mathcal{F}_D(\mathbf{x}) = \mathbf{M}\mathbf{x}, \text{ where}$$

$$\mathbf{M} = \frac{1}{\sqrt{p}} \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \dots & \omega^{p-1} \\ 1 & \omega^2 & \omega^4 & \dots & \omega^{2(p-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{p-1} & \omega^{2(p-1)} & \dots & \omega^{(p-1)^2} \end{bmatrix}. \quad (9)$$

The coefficient  $\omega$  is  $e^{-2\pi j/p}$ . Using the Euler’s rule and the  $i$ -th row of transformation  $\mathbf{M}$  be formulated as  $\psi(i) = [1, \cos(2\pi i/p), \dots, \cos(2\pi i(p-1)/p)]$ , we can rewrite  $\mathbf{M}$  as  $[\psi(0), \dots, \psi(p-1)]^\top / \sqrt{p}$ , and the window-wise similarity score is the sum of different groups:

$$\mathbf{S}_{(t_1, t_2)} = \frac{\mathbf{W}^\top}{\sqrt{p}} \left[ \begin{pmatrix} \psi(0) \\ \vdots \\ \mathbf{0} \end{pmatrix} + \dots + \begin{pmatrix} \mathbf{0} \\ \vdots \\ \psi(p-1) \end{pmatrix} \right] \begin{pmatrix} \varphi_0 \\ \vdots \\ \varphi_{p-1} \end{pmatrix}, \quad (10)$$

where the  $\mathbf{W}$  is the coefficient for decomposed and reorganized  $p$  sub-matrix and the  $\mathbf{0}$  is the all zero matrix.

We noticed that the pairwise attention scores  $\mathbf{S}'_{(t_1, t_2)}$  has been already calculated in Eq.(1). The target window-wise score  $\mathbf{S}_{(t_1, t_2)}$  can be acquired by performing convolution whose shape is larger than or equal to the periodic one in Eq.(10). Let  $\varphi = [\varphi_0, \varphi_1, \dots, \varphi_{p-1}]$ , we have:

$$\begin{aligned} \mathbf{S}_{(t_1, t_2)} &= \frac{\mathbf{W}^\top}{\sqrt{p}} [\psi(0)\varphi + \dots + \psi(p-1)\varphi] \\ &= \frac{\mathbf{W}^\top}{\sqrt{p}} [\text{avg}(\mathbf{S}') + \text{conv}_{\psi(1)}(\mathbf{S}') + \dots + \text{conv}_{\psi(p-1)}(\mathbf{S}')] \end{aligned} \quad (11)$$

where  $\text{conv}_{\psi(i)}(\cdot)$  represents the convolution operator with the kernel  $\psi(i)$ , with the same basis of DFT in Eq.(9). Thus, we can acquire the window-wise score by performing the convolution operator on the original attention score. Specially, for  $\psi(0)$ , we can use an all-one matrix as the kernel, which equals that we only select the  $\text{avg}(\cdot)$  operator.

#### 4.2. Establishing Ample Attention for Hybrid Encoding

Given the window-wise alignment defined on the similarity as Eq.(11), we introduce the Ample attention with alignment kernel and incorporate it with the vanilla attention to blend the window-wise and point-wise features. Specially, for the hybrid encoder block, we divide the attention heads

into two groups, including the vanilla attention heads and the Ample attention heads, and concatenate the results from these heads. The vanilla attention heads preserve point-wise similarity in Eq.(1), and the Ample attention is calculated with the alignment kernel formally defined as following.

#### 4.2.1. THE AMPLE ATTENTION

Recalling the vanilla Transformer’s attention mechanism, we still follows the  $\text{Softmax}(\cdot)$  design while replacing the pairwise similarity with the window-wise comparison. We applied generalized Parseval’s theorem (Hardy & Titchmarsh, 1931) between the pairwise  $\mathbf{S}'_{(t_1, t_2)}$  and window-wise  $\mathbf{S}_{(t_1, t_2)}$ , it reveals a linear connection on the sum of scores. It motivates us to leverage the  $\text{Conv2d}(\cdot)$  layer with learnable kernel, we initialized the kernel with a decomposing basis for fast convergence, then the distribution of linear coefficients  $\mathbf{W}$  is learned through convolution kernel. In this way, we can define the Ample Attention as:

$$\text{Attn} = \text{Softmax}(\text{Conv2d}_\psi(\mathbf{Q}\mathbf{K}^T)\mathbf{V}) \quad , \quad (12)$$

where the kernel  $\psi$  could be initialized by an exact type of transformation. If we only consider the term of  $\psi(0)$  in Eq.(11), we can perform  $\text{Avgpool2d}(\cdot)$  to replace the  $\text{Conv2d}_\psi(\cdot)$ .

#### 4.2.2. THE REDUCED KERNEL

In practice, as the Fourier basis involves imaginary calculation, bring difficulties for calculation, we simplify the window-wise alignment kernel with a Discrete Cosine Transform (DCT), which is a reduced form of DFT, containing real parts only. The discussion of the relationship and theoretical proofs are provided in Appendix I.5. Specifically, as As the previous derivation along with Eq.(11), we can initialize the kernel of  $\text{Conv2d}_\psi(\cdot)$  with the DCT basis, which is depicted as

$$\psi(i, j) = \begin{cases} \sqrt{\frac{1}{p}} \cos\left[\frac{(j + 0.5\pi)}{p} i\right] & i = 0 \\ \sqrt{\frac{2}{p}} \cos\left[\frac{(j + 0.5\pi)}{p} i\right] & i \neq 0 \end{cases} \quad . \quad (13)$$

Because of the learnable ability of the convolutional operators, the kernel will further adapt window alignment. With the reduced convolutional kernel, the ample attention calculates in a efficient way, and further computation costs are shown in Appendix C.

### 4.3. Denoising and Domain Conditioning

Integrating with the hybrid encoder, we establish the denoising network for diffusion process. As Eq.(3), for the  $n$ -th diffusion step, the denoising network iteratively predict the noise  $\epsilon_\theta$  from the noisy data  $\mathbf{X}_{(n)}$ .

To facilitate cross-domain generation, we construct the domain prompts inspired by TimeDP (Huang et al., 2025). Specifically, we construct the condition  $c$  with domain prompt  $c_d$ , learned prototypes  $c_p$  and step information  $c_t$ . And it is defined as

$$c = \text{Emb}_t(c_t) + \text{Emb}_{c_p}(c_p) + \text{Emb}_{c_d}(c_d) \quad , \quad (14)$$

where  $\text{Emb}_t(\cdot)$ ,  $\text{Emb}_{c_p}(\cdot)$  and  $\text{Emb}_{c_d}(\cdot)$  represent the embedding methods for these information. The condition  $c$  is injected by an adaptive Layer Norm Function as introduced by DiT (Peebles & Xie, 2023).

Injected with the condition  $c$ , the  $\mathbf{X}_{(n)}$  is processed by a stack of encoders and readout layer, which help to reconstruct  $\mathbf{X}_{(n-1)}$  from  $\mathbf{X}_{(n)}$ .

## 5. Experiments

In this section, we empirically demonstrate the effectiveness of Winformer on the 12 real-world datasets and further discuss its generating process through visualization. We also provide more experimental details and additional results in Appendix A to Appendix E, including hyper-parameter tests, computational costs, visualization figures and domain discrepancy analysis.

### 5.1. Setup: Cross-domain Time-series Generation

**(a) Datasets.** We conduct experiments on 12 real-world time-series datasets across four domains following the setting of TimeDP (Huang et al., 2025), including the time-series data of traffic flows, weather phenomena, industrial logs and financial records. All these datasets are reformatting into non-overlapping univariate sequence slices with the fixed length of 168. For cross-domain generation, time-series from diverse domains are mixed during training.

**(b) Baselines.** We compare our model with 6 representative SOTA methods for cross-domain time-series generation. These methods include GAN-base methods, such as TimeGAN (Yoon et al., 2019) and GT-GAN (Jeon et al., 2022), VAE-based methods, such as TimeVAE (Desai et al., 2021) and TimeVQVAE (Lee et al., 2023), and diffusion-based method, such as the newly released TimeDP (Huang et al., 2025) and Diffusion-TS (Yuan & Qiao, 2024). To ensure a fair comparison, we adopt partial related results reported by TimeDP (Huang et al., 2025).

**(c) Metrics.** We select two main metrics to evaluate the performance of generation by measuring the similarity between the distributions of the real and generated time-series. **Maximum Mean Discrepancy (MMD)** compares the discrepancy between two series after mapping into a high-dimensional feature space with a kernel function. **Kullback-Leibler Divergence (K-L)** measures the divergence between two probability distributions. The metric computation

Table 1. Results of generation results for sequence length 168. Best results are **bold** and second best results are underlined. Our method outperforms SOTA baselines in most of the datasets and achieve an increase of averagely 10.67% in MMD.

|                          |             | Winformer                | TimeDP                   | Diffusion-TS             | TimeGAN           | GT-GAN                   | TimeVAE            | TimeVQVAE         |
|--------------------------|-------------|--------------------------|--------------------------|--------------------------|-------------------|--------------------------|--------------------|-------------------|
| Maximum Mean Discrepancy | Electricity | <b>0.001</b> $\pm 0.002$ | <b>0.001</b> $\pm 0.001$ | <u>0.003</u> $\pm 0.002$ | 0.367 $\pm 0.255$ | 0.254 $\pm 0.166$        | 0.577 $\pm 0.006$  | 0.152 $\pm 0.024$ |
|                          | Solar       | <b>0.035</b> $\pm 0.004$ | <u>0.041</u> $\pm 0.011$ | <u>0.050</u> $\pm 0.012$ | 0.628 $\pm 0.053$ | 0.578 $\pm 0.039$        | 0.353 $\pm 0.014$  | 0.437 $\pm 0.020$ |
|                          | Wind        | <u>0.034</u> $\pm 0.014$ | <b>0.025</b> $\pm 0.017$ | 0.035 $\pm 0.006$        | 0.213 $\pm 0.017$ | 0.170 $\pm 0.040$        | 0.170 $\pm 0.004$  | 0.131 $\pm 0.014$ |
|                          | Traffic     | <b>0.071</b> $\pm 0.005$ | <u>0.083</u> $\pm 0.034$ | 0.111 $\pm 0.031$        | 0.567 $\pm 0.057$ | 0.538 $\pm 0.078$        | 0.218 $\pm 0.007$  | 0.213 $\pm 0.016$ |
|                          | Taxi        | <b>0.085</b> $\pm 0.010$ | <u>0.095</u> $\pm 0.023$ | 0.131 $\pm 0.014$        | 0.275 $\pm 0.054$ | 0.319 $\pm 0.032$        | 0.139 $\pm 0.007$  | 0.128 $\pm 0.004$ |
|                          | Pedestrian  | <b>0.040</b> $\pm 0.008$ | <u>0.044</u> $\pm 0.020$ | 0.071 $\pm 0.019$        | 0.090 $\pm 0.030$ | 0.112 $\pm 0.019$        | 0.065 $\pm 0.002$  | 0.067 $\pm 0.007$ |
|                          | Air         | <b>0.011</b> $\pm 0.002$ | <b>0.011</b> $\pm 0.003$ | <u>0.022</u> $\pm 0.011$ | 0.120 $\pm 0.045$ | 0.211 $\pm 0.041$        | 0.089 $\pm 0.016$  | 0.028 $\pm 0.002$ |
|                          | Temperature | <u>0.230</u> $\pm 0.021$ | <b>0.219</b> $\pm 0.022$ | <u>0.241</u> $\pm 0.049$ | 0.926 $\pm 0.042$ | 0.809 $\pm 0.081$        | 1.002 $\pm 0.014$  | 0.323 $\pm 0.008$ |
|                          | Rain        | <b>0.036</b> $\pm 0.016$ | <u>0.057</u> $\pm 0.039$ | 0.079 $\pm 0.058$        | 0.329 $\pm 0.285$ | 0.111 $\pm 0.109$        | 0.292 $\pm 0.019$  | 0.074 $\pm 0.007$ |
|                          | NN5         | <b>0.147</b> $\pm 0.008$ | <u>0.164</u> $\pm 0.010$ | 0.186 $\pm 0.043$        | 0.874 $\pm 0.088$ | 0.632 $\pm 0.074$        | 0.821 $\pm 0.061$  | 0.327 $\pm 0.012$ |
|                          | Fred-MD     | <b>0.002</b> $\pm 0.001$ | <b>0.002</b> $\pm 0.001$ | <u>0.006</u> $\pm 0.002$ | 0.043 $\pm 0.021$ | 0.133 $\pm 0.102$        | 0.059 $\pm 0.008$  | 0.008 $\pm 0.002$ |
|                          | Exchange    | <b>0.137</b> $\pm 0.012$ | <u>0.151</u> $\pm 0.024$ | <u>0.206</u> $\pm 0.035$ | 0.530 $\pm 0.154$ | 0.475 $\pm 0.116$        | 0.543 $\pm 0.149$  | 0.342 $\pm 0.050$ |
| K-L Divergence           | Electricity | <b>0.008</b> $\pm 0.010$ | <u>0.012</u> $\pm 0.016$ | 0.315 $\pm 0.247$        | 0.488 $\pm 0.175$ | 0.407 $\pm 0.079$        | 0.734 $\pm 0.023$  | 0.280 $\pm 0.051$ |
|                          | Solar       | <b>0.013</b> $\pm 0.005$ | <u>0.016</u> $\pm 0.005$ | 0.066 $\pm 0.055$        | 0.612 $\pm 0.447$ | 0.120 $\pm 0.041$        | 0.260 $\pm 0.016$  | 0.865 $\pm 0.108$ |
|                          | Wind        | 0.202 $\pm 0.044$        | <u>0.152</u> $\pm 0.034$ | 0.548 $\pm 0.372$        | 1.924 $\pm 1.233$ | <b>0.107</b> $\pm 0.016$ | 0.484 $\pm 0.015$  | 0.483 $\pm 0.066$ |
|                          | Traffic     | <u>0.011</u> $\pm 0.002$ | <b>0.009</b> $\pm 0.003$ | 0.120 $\pm 0.074$        | 1.305 $\pm 0.320$ | 1.409 $\pm 0.251$        | 0.211 $\pm 0.014$  | 0.178 $\pm 0.026$ |
|                          | Taxi        | <b>0.005</b> $\pm 0.003$ | <u>0.011</u> $\pm 0.004$ | 0.075 $\pm 0.034$        | 0.650 $\pm 0.180$ | 0.950 $\pm 0.197$        | 0.110 $\pm 0.020$  | 0.110 $\pm 0.026$ |
|                          | Pedestrian  | <b>0.009</b> $\pm 0.004$ | <u>0.014</u> $\pm 0.010$ | 0.133 $\pm 0.069$        | 0.417 $\pm 0.181$ | 0.411 $\pm 0.096$        | 0.065 $\pm 0.005$  | 0.405 $\pm 0.051$ |
|                          | Air         | <b>0.026</b> $\pm 0.011$ | <u>0.027</u> $\pm 0.016$ | 0.106 $\pm 0.079$        | 0.348 $\pm 0.093$ | 0.578 $\pm 0.049$        | 0.164 $\pm 0.012$  | 0.054 $\pm 0.012$ |
|                          | Temperature | <u>0.176</u> $\pm 0.027$ | <b>0.171</b> $\pm 0.073$ | 0.342 $\pm 0.131$        | 8.892 $\pm 2.681$ | 3.174 $\pm 2.685$        | 2.183 $\pm 0.110$  | 0.735 $\pm 0.066$ |
|                          | Rain        | <b>0.011</b> $\pm 0.003$ | <u>0.013</u> $\pm 0.012$ | 0.061 $\pm 0.072$        | 0.506 $\pm 0.174$ | 0.432 $\pm 0.099$        | 0.160 $\pm 0.022$  | 0.047 $\pm 0.018$ |
|                          | NN5         | <b>0.045</b> $\pm 0.007$ | <u>0.054</u> $\pm 0.014$ | 0.165 $\pm 0.076$        | 4.928 $\pm 4.112$ | 1.386 $\pm 0.520$        | 1.337 $\pm 0.220$  | 1.063 $\pm 0.274$ |
|                          | Fred-MD     | <b>0.201</b> $\pm 0.014$ | <u>0.203</u> $\pm 0.035$ | 0.835 $\pm 0.554$        | 0.512 $\pm 0.290$ | 0.380 $\pm 0.070$        | 0.346 $\pm 0.041$  | 0.831 $\pm 0.077$ |
|                          | Exchange    | <b>1.621</b> $\pm 0.126$ | <u>1.866</u> $\pm 0.132$ | 2.337 $\pm 0.714$        | 8.861 $\pm 3.397$ | 7.201 $\pm 4.380$        | 10.404 $\pm 1.434$ | 5.052 $\pm 1.385$ |
| Count                    | 19          | 7                        | 0                        | 0                        | 1                 | 0                        | 0                  |                   |

is stated in Appendix G.1. MMD measures global distribution, while KL is sensitive to local point-wise noise. These two metrics measure the results in different aspect. In addition to intra-domain comparison, we also evaluated the domain discrepancy for inter-domain comparison, with full results shown in Appendix E.

**(d) Implementation.** As analyzed in Section 4, the stride length of window-alignment is a hyper-parameter, and we choose 25 in our main experiments, which is suitable due to it is larger than common periodic cycles in real-world datasets, such as 4, 6, 12 and 24. We further explored the effectiveness with different stride length and kernel type of window-alignment in Appendix A. The number of DiT encoder layers is set to 6, and the hidden size is 512. The learning rate is set to  $5 \times 10^{-5}$  with 1,000 warm-up steps. For the diffusion process, we use 200 steps adding noise to the series or reconstructing them. The experiments are repeated 5 times with seeds spanning from 2021 to 2025. More details about implementation can be found in the Appendix G.

## 5.2. Main Results

In this experiment, we evaluate our methods with 6 representative baselines in Table 1. Our method achieves the best performance in 10 out of the 12 datasets measuring with maximum mean discrepancy, which indicates that the time-series generated by our model conforms to the original sequence better than other competitive methods. Considering all of the 12 datasets, our method averagely decreases the MMD with 10.67% compared to TimeDP, which is the SOTA method of cross-domain generation.

From the detailed results on each domain, we can find that the Winformer demonstrates especially stronger ability on datasets with strong periodicity, such as solar and traffic. On the contrary, the ability of Winformer lags slightly behind on datasets exhibiting a stronger tendency toward trendiness rather than periodicity, such as wind and temperature. For further analysis on periodicity capturing, we visualized an example of the generated series with both the original signal and the frequency spectrum as shown in Fig. 4. It's obvious that time-series signals generated by our method are more

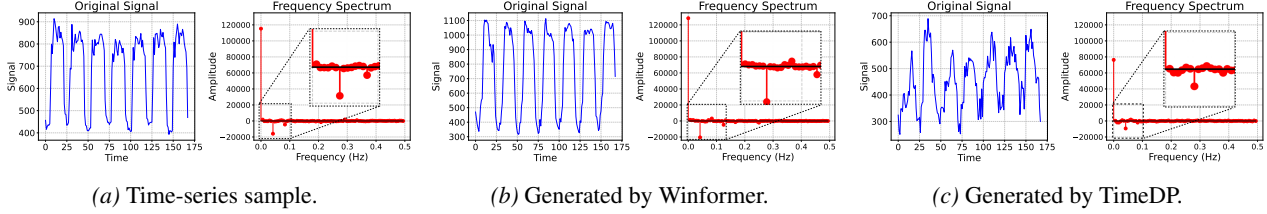


Figure 4. Visualization of generated series, comparing on original signal and frequency spectrum. Through comparison, we found that the samples generated by our model exhibit better feature learning in the frequency domain.

Table 2. Results for ablation study on point-wise, patch-wise and our window-wise alignment. The best results of each line are **bold**, and our window-wise alignment outperforms other methods with 15 best scores.

| Alignment Type | Window-wise Alignment    |                          | Patch-wise Alignment     |                          | Point-wise Alignment     |                          |
|----------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|
| Metric         | MMD                      | K-L                      | MMD                      | K-L                      | MMD                      | K-L                      |
| Electricity    | <b>0.001</b> $\pm 0.002$ | <b>0.008</b> $\pm 0.010$ | 0.002 $\pm 0.005$        | 0.011 $\pm 0.005$        | 0.002 $\pm 0.004$        | 0.030 $\pm 0.028$        |
| Solar          | <b>0.035</b> $\pm 0.004$ | <b>0.013</b> $\pm 0.005$ | <b>0.035</b> $\pm 0.003$ | 0.016 $\pm 0.007$        | <b>0.035</b> $\pm 0.003$ | 0.017 $\pm 0.011$        |
| Wind           | 0.034 $\pm 0.014$        | 0.202 $\pm 0.044$        | <b>0.033</b> $\pm 0.010$ | <b>0.176</b> $\pm 0.005$ | 0.034 $\pm 0.010$        | 0.186 $\pm 0.026$        |
| Traffic        | <b>0.071</b> $\pm 0.005$ | 0.011 $\pm 0.002$        | 0.109 $\pm 0.012$        | 0.012 $\pm 0.010$        | 0.073 $\pm 0.008$        | <b>0.010</b> $\pm 0.003$ |
| Taxi           | <b>0.085</b> $\pm 0.010$ | 0.005 $\pm 0.003$        | 0.195 $\pm 0.000$        | 0.014 $\pm 0.000$        | 0.088 $\pm 0.011$        | <b>0.004</b> $\pm 0.002$ |
| Pedestrian     | 0.040 $\pm 0.008$        | <b>0.009</b> $\pm 0.004$ | <b>0.034</b> $\pm 0.014$ | 0.011 $\pm 0.135$        | 0.041 $\pm 0.010$        | 0.012 $\pm 0.009$        |
| Air            | <b>0.011</b> $\pm 0.002$ | <b>0.026</b> $\pm 0.011$ | 0.034 $\pm 0.001$        | 0.045 $\pm 0.016$        | 0.012 $\pm 0.001$        | 0.028 $\pm 0.010$        |
| Temperature    | 0.230 $\pm 0.021$        | <b>0.176</b> $\pm 0.027$ | <b>0.214</b> $\pm 0.004$ | 0.177 $\pm 0.009$        | 0.217 $\pm 0.026$        | <b>0.176</b> $\pm 0.042$ |
| Rain           | <b>0.036</b> $\pm 0.016$ | <b>0.011</b> $\pm 0.003$ | 0.052 $\pm 0.027$        | 0.039 $\pm 0.033$        | 0.039 $\pm 0.024$        | <b>0.011</b> $\pm 0.004$ |
| NN5            | 0.147 $\pm 0.008$        | <b>0.045</b> $\pm 0.007$ | <b>0.146</b> $\pm 0.015$ | 0.080 $\pm 0.006$        | 0.152 $\pm 0.008$        | 0.048 $\pm 0.013$        |
| Fred-MD        | <b>0.002</b> $\pm 0.001$ | 0.201 $\pm 0.014$        | 0.002 $\pm 0.005$        | <b>0.199</b> $\pm 0.003$ | <b>0.002</b> $\pm 0.001$ | 0.218 $\pm 0.048$        |
| Exchange       | <b>0.137</b> $\pm 0.012$ | 1.621 $\pm 0.126$        | 0.139 $\pm 0.011$        | <b>1.595</b> $\pm 0.023$ | 0.139 $\pm 0.014$        | 1.625 $\pm 0.115$        |
| Count          | <b>15</b>                |                          | 8                        |                          | 6                        |                          |

similar to the original data. By observing the frequency spectrum, we can infer that our window-to-window alignment method can extract more detailed periodic patterns than TimeDP, which probably explains why our method outperforms SOTA baselines.

The experimental phenomenon is consistent with our understanding and analysis of window-to-window alignment. Since periodicity is widely presented in real-world time-series datasets, our Winformer achieves superior performance across most scenarios.

### 5.3. Ablation Study

As the ablation results in Table 2, we evaluated the performance with our **window-wise alignment**, and traditional methods, which modeling the time-series in **patch-wise alignment** and **point-wise alignment**. The window-wise alignment achieves the best performance on most of the datasets. And patch-wise alignment shows slightly advantages in dataset wind considering both KL and MMD, which is because the periodic features in this dataset is relatively weak. However, our model still has obvious superiority in most scenarios, because periodicity is widely present in

real-world time-series data. More visualized ablation results are shown in Appendix D.2.

We provide results comparing random initiate kernel and our method, with results in Fig. 7, showing our advancement in full frequency modeling. We further explore how the kernel changes during the training process in Sec. 6, showing the random initialized kernel tends to become an Avg-Pooling, which is a special case within our proposed framework.

## 6. Discussion

In this section, we further analyze the intermediate results and phenomena to understand how our method enhances the generation process.

**Q1: How does the Ample attention help the diffusion model recovering the time-series patterns?** We conduct experiments on a synthetic time-series data as Figure 5(a), and it contains sinusoidal signals with different cycle periods. The largest period is 50, and the smallest is 5. We evaluate the proposed model on the synthetic data and draw the product score’s visualization between input **Q** and **K** in Eq.(1). Due to space limit, we place the score matrix of the

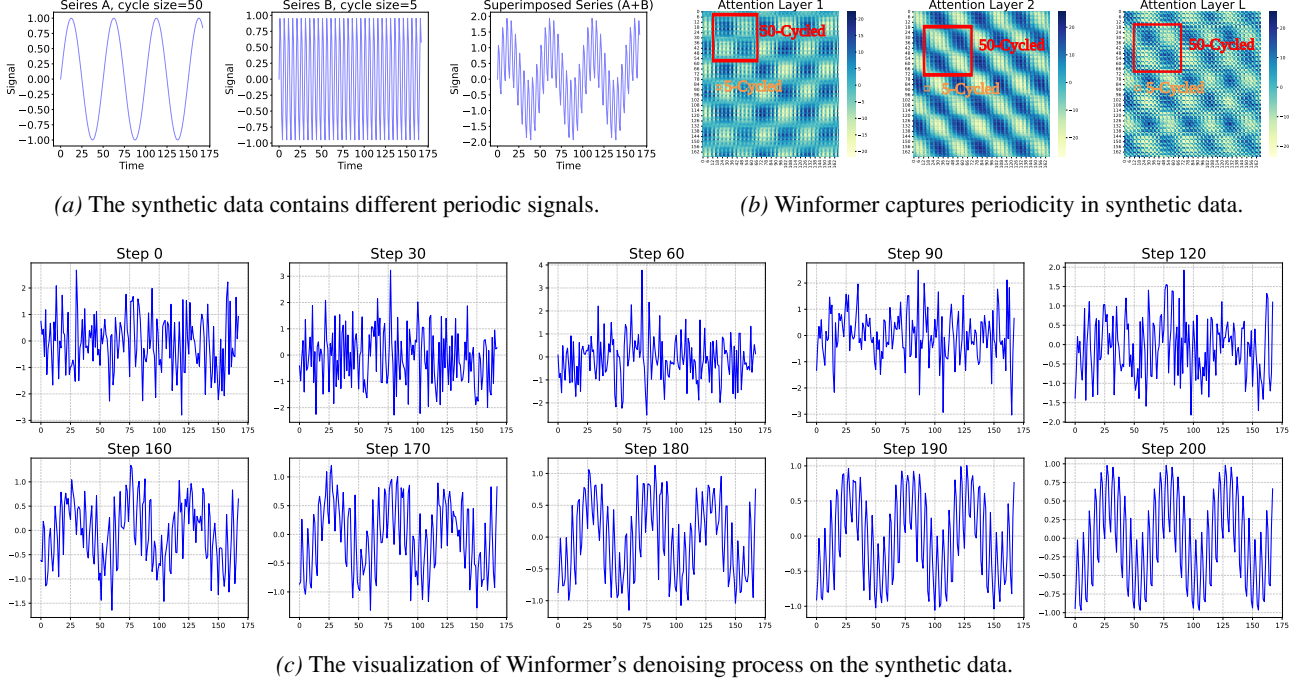


Figure 5. Intuitive validation on synthetic data. On the synthetic data shown in (a), the attention map produced by Winformer exhibits the pattern illustrated in (b), indicating that periodicity is one of the primary characteristics reflected in the attention map. The process of generating the final sequence is illustrated in (c), where coarse-grained long-periods (50-cycled) emerge first, followed by fine-grained short-period (5-cycled).

first two layers and the last one (more visualization figures are in Appendix D), where the darker area in heatmap indicates closer time-series interactions. The swapped color stripes reveals distinct two kinds of cycle patterns: the smaller orange grid is 5-cycled and the bigger red one is 50-cycled. Thus, we could leverage the Ample attention to capture the periodic interactions for better recovering.

**Q2: How does the Winformer architecture utilize the periodic information during the denoising process?** We visualized the denoising process in Figure 5(c), and find out that the longer cycle firstly emerges then the rest during the denoising process. Taking a concrete example at step 140, it presents a rough cycle with period 50, then the cycle period shrinks to 5 at step 180. The Winformer appears to first identify major periodic patterns that have a wider impact, then incorporate more detailed periodic patterns. This adaptive nature enables Winformer to have generalized window alignment capability in denoising process, particularly when applied to the cross-domain cases.

**Q3: How does the aligning kernel change during the training process?** To understand what the model learned within the aligning kernel, we have explored the changes of the kernel during the training process, considering on two kinds of initialization for comparison. **Considering the randomly initialized kernel weights**, the Fig. 6(a) and Fig. 6(b) represent the kernel weights before and after training with

randomly initialized weights. It can be observed that the values in the weight matrix tend to be evenly distributed, making it similar to the ‘kernel’ of an average pooling layer, as the weights for each position are nearly equal. As we discussed in Eq.(11), the  $\text{avg}(\cdot)$  represents the  $\psi(0)$ , which is a special form of the aligning kernel. Additional results with the kernel implemented by the averaging pooling are provided in Appendix A. **If initialize with the DCT-based kernel weights**, the visualizations for kernel weights are shown in Fig. 6(c) and Fig. 6(d). It indicates that the weights change slightly, because the DCT basis shows advantages in periodicity capturing and provides sufficient guidance for denoising. Thus, the aligning kernel persists its periodicity capturing ability during training.

## 7. Conclusion

We address the challenge of periodicity alignment for cross-domain time-series generation by reforming the attention mechanism in window-wise alignment. Specifically, we propose Winformer, a diffusion framework built on window-wise modeling, which replaces pairwise similarity in vanilla attention with window-to-window comparison via time-frequency analysis. With theoretical deduction, the window-wise alignment is reducible to learnable convolutions, enabling effective implementation. Winformer shows advanced performance by achieving an average 10.67% MMD

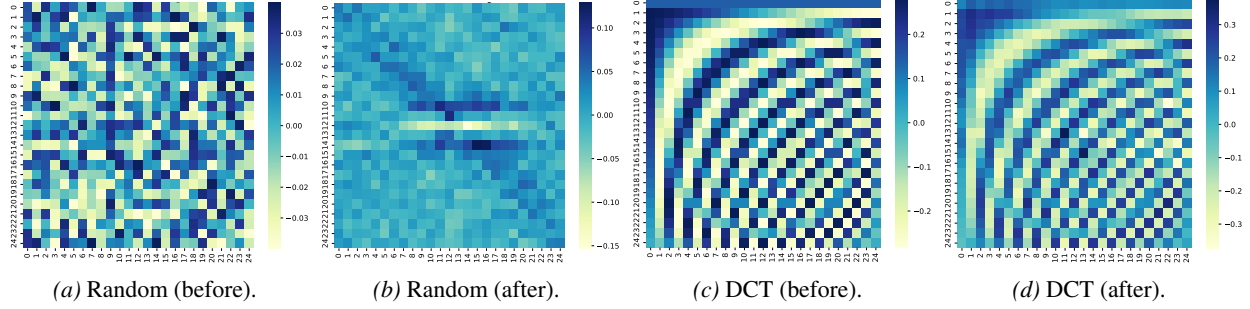


Figure 6. Visualization of the kernels for window-wise alignment, with random initialization or with DCT-based initialization. The visualization compares the kernel before and after training. It can be observed that kernels initialized randomly tend to converge toward average pooling after training, whereas kernels initialized with DCT largely preserve the original DCT bases after training.

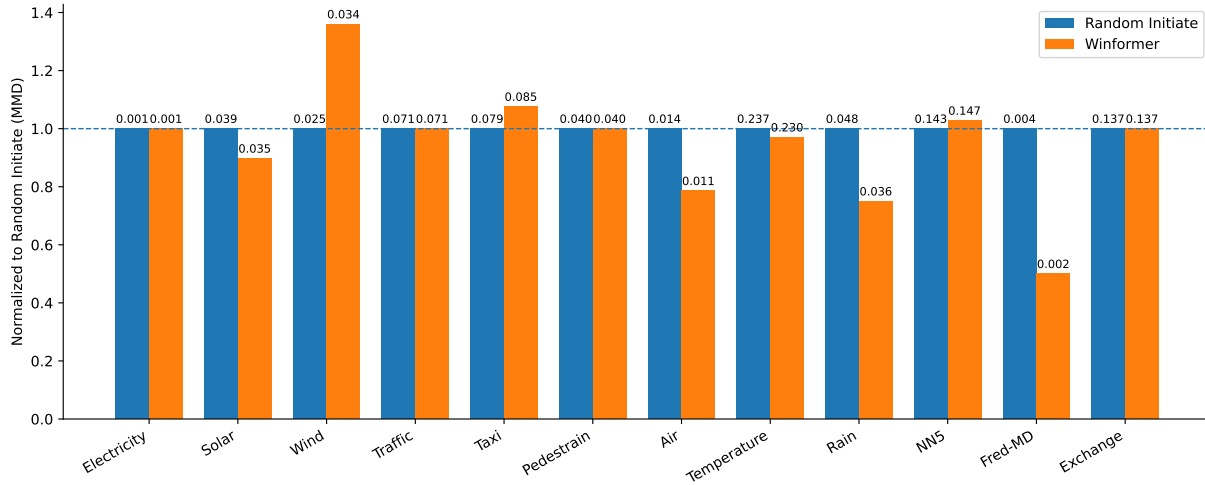


Figure 7. Results for ablation study on ablation study for kernel initialization. As the random initialized kernel tend to converge toward average pooling after training, which is a special case of our framework, it contains strong performance in a certain of datasets.

improvement over SOTA baselines on 12 real-world datasets. Visualization analysis and ablation studies verified the effectiveness of our window-wise alignment. Future works will focus on better conditioning, multi-modal integration, and diffusion efficiency optimization.

## 8. Limitations

Although Winformer achieves competitive time-series forecasting performance, it has limitation as its applicability boundary remains unexplored. First, the model degrades on trend-dominated data like wind and temperature data, while no usage guidance is provided for practitioners. We provide a simple idea of integrating our model with trend decomposition in Appendix F.1, showing it has the potential to further improve performance on datasets with prominent trends. Second, this work leaves the window-wise alignment extension for multivariate time-series scenarios uninvestigated. We provide a preliminary idea of combining GCN for multi-variable series in Appendix F.2. Third, it's diffi-

cult for hyper-parameter selecting for long seasonal cycles. However, the hyper-parameter  $s$  can be set to a large enough value, as it could backward compatible with smaller cycles.

## Acknowledgement

This work was supported by the grants from the Natural Science Foundation of China (62572035, 62225202), and Beijing Natural Science Foundation (L248032). Thanks for the computing infrastructure provided by Beijing Advanced Innovation Center for Big Data and Brain Computing. Sponsored by CAAI-MindSpore Open Fund, developed on OpenI Community. Jianxin Li is the corresponding author.

## Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

## References

- Alaa, A. M., Chan, A. J., and van der Schaar, M. Generative time-series modeling with fourier flows. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL <https://openreview.net/forum?id=PpshD0AXfA>.
- Alcaraz, J. M. L. and Strodthoff, N. Diffusion-based time series imputation and forecasting with structured state space models. *Trans. Mach. Learn. Res.*, 2023, 2023. URL <https://openreview.net/forum?id=hHiIbk7ApW>.
- Cao, D., Ye, W., Zhang, Y., and Liu, Y. Timedit: General-purpose diffusion transformers for time series foundation model, 2025. URL <https://arxiv.org/abs/2409.02322>.
- Desai, A., Freeman, C., Wang, Z., and Beaver, I. Timevae: A variational auto-encoder for multivariate time series generation. *CoRR*, abs/2111.08095, 2021. URL <https://arxiv.org/abs/2111.08095>.
- Devlin, J., Chang, M., Lee, K., and Toutanova, K. BERT: pre-training of deep bidirectional transformers for language understanding. In Burstein, J., Doran, C., and Solorio, T. (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, pp. 4171–4186. Association for Computational Linguistics, 2019. doi: 10.18653/V1/N19-1423. URL <https://doi.org/10.18653/v1/n19-1423>.
- Duhamel, P. and Vetterli, M. Fast fourier transforms: a tutorial review and a state of the art. *Signal processing*, 19(4):259–299, 1990.
- Feng, S., Miao, C., Zhang, Z., and Zhao, P. Latent diffusion transformer for probabilistic time series forecasting. In *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence, AAAI’24/IAAI’24/EAAI’24*. AAAI Press, 2024. ISBN 978-1-57735-887-9. doi: 10.1609/aaai.v38i11.29085. URL <https://doi.org/10.1609/aaai.v38i11.29085>.
- Ge, Y., Li, J., Zhao, Y., Wen, H., Li, Z., Qiu, M., Li, H., Jin, M., and Pan, S. T2S: high-resolution time series generation with text-to-series diffusion models. *CoRR*, abs/2505.02417, 2025. doi: 10.48550/ARXIV.2505.02417. URL <https://doi.org/10.48550/arXiv.2505.02417>.
- Günther, M., Ong, J., Mohr, I., Abdesslem, A., Abel, T., Akram, M. K., Guzman, S., Mastrapas, G., Sturua, S., Wang, B., Werk, M., Wang, N., and Xiao, H. Jina embeddings 2: 8192-token general-purpose text embeddings for long documents. *CoRR*, abs/2310.19923, 2023. doi: 10.48550/ARXIV.2310.19923. URL <https://doi.org/10.48550/arXiv.2310.19923>.
- Hardy, G. H. and Titchmarsh, E. A note on parseval’s theorem for fourier transforms. *Journal of the London Mathematical Society*, s1-6(1):44–48, 01 1931. ISSN 0024-6107. doi: 10.1112/jlms/s1-6.1.44. URL <https://doi.org/10.1112/jlms/s1-6.1.44>.
- Huang, Y., Xu, C., Wu, Y., Li, W., and Bian, J. Timedp: Learning to generate multi-domain time series with domain prompts. In Walsh, T., Shah, J., and Kolter, Z. (eds.), *AAAI-25, Sponsored by the Association for the Advancement of Artificial Intelligence, February 25 - March 4, 2025, Philadelphia, PA, USA*, pp. 17520–17527. AAAI Press, 2025. doi: 10.1609/AAAI.V39I17.33926. URL <https://doi.org/10.1609/aaai.v39i17.33926>.
- Jeon, J., Kim, J., Song, H., Cho, S., and Park, N. GT-GAN: general purpose time series synthesis with generative adversarial networks. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022.
- Kim, T., Kim, J., Tae, Y., Park, C., Choi, J., and Choo, J. Reversible instance normalization for accurate time-series forecasting against distribution shift. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. URL <https://openreview.net/forum?id=cGDakQo1C0p>.
- Kong, L., Dong, J., Ge, J., Li, M., and Pan, J. Efficient frequency domain-based transformers for high-quality image deblurring. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023, Vancouver, BC, Canada, June 17-24, 2023*, pp. 5886–5895. IEEE, 2023. doi: 10.1109/CVPR52729.2023.00570. URL <https://doi.org/10.1109/CVPR52729.2023.00570>.
- Lee, D., Malacarne, S., and Aune, E. Vector quantized time series generation with a bidirectional prior model. In Ruiz, F. J. R., Dy, J. G., and van de Meent, J. (eds.), *International Conference on Artificial Intelligence and Statistics, 25-27 April 2023, Palau de Congressos, Valencia, Spain*, volume 206 of *Proceedings*

- of *Machine Learning Research*, pp. 7665–7693. PMLR, 2023. URL <https://proceedings.mlr.press/v206/lee23d.html>.
- Liu, X., Hu, J., Li, Y., Diao, S., Liang, Y., Hooi, B., and Zimmermann, R. Unitime: A language-empowered unified model for cross-domain time series forecasting. In *Proceedings of the ACM Web Conference 2024*, 2024.
- Ma, X., Hong, X., Lin, M., Zhang, H., Li, W., and Lu, S. Domain fusion controllable generalization for cross-domain time series forecasting from multi-domain integrated distribution, 2025. URL <https://arxiv.org/abs/2412.03068>.
- McGillem, C. D. and Cooper, G. R. Continuous and discrete signal and system analysis. (*No Title*), 1991.
- Mikolov, T., Chen, K., Corrado, G., and Dean, J. Efficient estimation of word representations in vector space. In Bengio, Y. and LeCun, Y. (eds.), *1st International Conference on Learning Representations, ICLR 2013, Scottsdale, Arizona, USA, May 2-4, 2013, Workshop Track Proceedings*, 2013. URL <http://arxiv.org/abs/1301.3781>.
- Nie, Y., Nguyen, N. H., Sinthong, P., and Kalagnanam, J. A time series is worth 64 words: Long-term forecasting with transformers. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. URL <https://openreview.net/forum?id=Jbdc0vTOcol>.
- Ogasawara, E., Martinez, L. C., de Oliveira, D., Zimbrão, G., Pappa, G. L., and Mattoso, M. Adaptive normalization: A novel data normalization approach for non-stationary time series. *The 2010 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8, 2010. URL <https://api.semanticscholar.org/CorpusID:5757527>.
- Peebles, W. and Xie, S. Scalable diffusion models with transformers. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*, pp. 4172–4182. IEEE, 2023. doi: 10.1109/ICCV51070.2023.00387. URL <https://doi.org/10.1109/ICCV51070.2023.00387>.
- Piao, X., Chen, Z., Murayama, T., Matsubara, Y., and Sakurai, Y. Fredformer: Frequency debiased transformer for time series forecasting. In Baeza-Yates, R. and Bonchi, F. (eds.), *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2024, Barcelona, Spain, August 25-29, 2024*, pp. 2400–2410. ACM, 2024. doi: 10.1145/3637528.3671928. URL <https://doi.org/10.1145/3637528.3671928>.
- Tashiro, Y., Song, J., Song, Y., and Ermon, S. CSDI: conditional score-based diffusion models for probabilistic time series imputation. In Ranzato, M., Beygelzimer, A., Dauphin, Y. N., Liang, P., and Vaughan, J. W. (eds.), *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pp. 24804–24816, 2021.
- Ulyanov, D., Vedaldi, A., and Lempitsky, V. S. Instance normalization: The missing ingredient for fast stylization. *ArXiv*, abs/1607.08022, 2016. URL <https://api.semanticscholar.org/CorpusID:16516553>.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. In Guyon, I., von Luxburg, U., Bengio, S., Wallach, H. M., Fergus, R., Vishwanathan, S. V. N., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pp. 5998–6008, 2017.
- Wang, L. and Li, C. Adjacent neighborhood transformer-based diffusion model for anomaly detection under incomplete industrial data sources. In Nejdli, W., Auer, S., Cha, M., Moens, M., and Najork, M. (eds.), *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining, WSDM 2025, Hannover, Germany, March 10-14, 2025*, pp. 577–585. ACM, 2025. doi: 10.1145/3701551.3703565. URL <https://doi.org/10.1145/3701551.3703565>.
- Wu, H., Xu, J., Wang, J., and Long, M. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In Ranzato, M., Beygelzimer, A., Dauphin, Y. N., Liang, P., and Vaughan, J. W. (eds.), *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pp. 22419–22430, 2021.
- Yoon, J., Jarrett, D., and van der Schaar, M. Time-series generative adversarial networks. In Wallach, H. M., Larochelle, H., Beygelzimer, A., d’Alché-Buc, F., Fox, E. B., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pp. 5509–5519, 2019.
- Yuan, X. and Qiao, Y. Diffusion-ts: Interpretable diffusion for general time series generation. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=4hlapFjO99>.

Zhang, B., Luo, F., He, Y., and Ranzi, G. Gt-nilm: A generative, transferable non-intrusive load monitoring system based on conditional diffusion models and convolutional neural networks. *IEEE Transactions on Consumer Electronics*, pp. 1–1, 2025. doi: 10.1109/TCE.2025.3577139.

Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., and Zhang, W. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational Advances in Artificial Intelligence, EAAI 2021, Virtual Event, February 2-9, 2021*, pp. 11106–11115. AAAI Press, 2021. doi: 10.1609/AAAI.V35I12.17325. URL <https://doi.org/10.1609/aaai.v35i12.17325>.

Zhou, T., Ma, Z., Wen, Q., Wang, X., Sun, L., and Jin, R. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvári, C., Niu, G., and Sabato, S. (eds.), *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, volume 162 of *Proceedings of Machine Learning Research*, pp. 27268–27286. PMLR, 2022. URL <https://proceedings.mlr.press/v162/zhou22g.html>.

## A. Hyper-parameter Study

In this experiment, we evaluated the performance with different kernel types and sizes. The results are shown in Table 3, and the best model, which adapts a 25-size Conv2d layer as the aligning kernel, whose results are also selected to be shown in Table 1 competing with SOTA baselines.

**For kernel types**, we apply two types of kernels. The Conv2d kernel is implemented with a 2D convolution neural network, whose kernel is initialized with the DCT basis. The Avgpool2d kernel is implemented with a 2D average pooling operator. The results show that Conv2d performs better than Avgpool2d, and both operators show effectiveness compared to the model without any kernel. We can find that DCT kernel performs better than Avg pooling because DCT kernel could capture more periodicity information, while DCT achieves  $8+8+9=25$  best counts and Avg achieves  $4+9+5=18$  best counts. Thus, in this paper, we mainly consider the DCT kernels. By the way, Avg Pooling is more efficient, so it's more suitable with limited calculation consumption. **For kernel sizes**, we select three sizes, including 7, 13 and 25, which are odd numbers to facilitate the convolution operator. The results show that, for Conv2d kernel, model with the size of 25 performs best because it contains the cycle up to 24, which is a common cycle for time-series data. The optimal 25-size kernel in the paper works because it includes typical cycles (4, 6, 12, 24) without overcomplicating computation.

Table 3. Results for hyper-parameters study . Best results of each line are **bold**. The best model uses 2D Convolution layer with the kernel size of 25, and it achieves best performance on 9 lines in total, which is also selected as our final model shown in Table 1.

| # Kernel Type            |             | Window-wise align by Conv2d kernel |                          |                          | Window-wise align by Avgpool2d kernel |                          |                          |
|--------------------------|-------------|------------------------------------|--------------------------|--------------------------|---------------------------------------|--------------------------|--------------------------|
| # Kernel Size            |             | 7                                  | 13                       | 25                       | 7                                     | 13                       | 25                       |
| Maximum Mean Discrepancy | Electricity | <b>0.001</b> $\pm 0.001$           | 0.002 $\pm 0.003$        | <b>0.001</b> $\pm 0.002$ | 0.002 $\pm 0.003$                     | 0.002 $\pm 0.003$        | 0.002 $\pm 0.003$        |
|                          | Solar       | 0.035 $\pm 0.001$                  | 0.035 $\pm 0.002$        | 0.035 $\pm 0.004$        | <b>0.033</b> $\pm 0.005$              | <b>0.033</b> $\pm 0.004$ | <b>0.033</b> $\pm 0.004$ |
|                          | Wind        | 0.035 $\pm 0.015$                  | <b>0.033</b> $\pm 0.011$ | 0.034 $\pm 0.014$        | <b>0.033</b> $\pm 0.010$              | <b>0.033</b> $\pm 0.010$ | 0.037 $\pm 0.013$        |
|                          | Traffic     | 0.074 $\pm 0.007$                  | 0.074 $\pm 0.003$        | <b>0.071</b> $\pm 0.005$ | 0.072 $\pm 0.003$                     | <b>0.071</b> $\pm 0.003$ | 0.074 $\pm 0.005$        |
|                          | Taxi        | <b>0.080</b> $\pm 0.015$           | 0.086 $\pm 0.012$        | 0.085 $\pm 0.010$        | 0.084 $\pm 0.013$                     | 0.083 $\pm 0.012$        | 0.082 $\pm 0.011$        |
|                          | Pedestrian  | <b>0.040</b> $\pm 0.011$           | 0.041 $\pm 0.010$        | <b>0.040</b> $\pm 0.008$ | 0.042 $\pm 0.009$                     | 0.041 $\pm 0.010$        | <b>0.040</b> $\pm 0.011$ |
|                          | Air         | 0.014 $\pm 0.004$                  | 0.013 $\pm 0.001$        | <b>0.011</b> $\pm 0.002$ | 0.013 $\pm 0.001$                     | 0.013 $\pm 0.001$        | 0.012 $\pm 0.002$        |
|                          | Temperature | 0.226 $\pm 0.022$                  | <b>0.219</b> $\pm 0.027$ | 0.230 $\pm 0.021$        | 0.220 $\pm 0.026$                     | <b>0.219</b> $\pm 0.024$ | 0.229 $\pm 0.023$        |
|                          | Rain        | <b>0.033</b> $\pm 0.012$           | 0.050 $\pm 0.046$        | 0.036 $\pm 0.016$        | 0.038 $\pm 0.024$                     | 0.037 $\pm 0.019$        | 0.036 $\pm 0.016$        |
|                          | NN5         | 0.149 $\pm 0.005$                  | 0.151 $\pm 0.007$        | <b>0.147</b> $\pm 0.008$ | 0.153 $\pm 0.006$                     | 0.154 $\pm 0.006$        | 0.154 $\pm 0.007$        |
|                          | Fred-MD     | <b>0.002</b> $\pm 0.001$           | <b>0.002</b> $\pm 0.001$ | <b>0.002</b> $\pm 0.001$ | <b>0.002</b> $\pm 0.001$              | <b>0.002</b> $\pm 0.001$ | <b>0.002</b> $\pm 0.001$ |
|                          | Exchange    | <b>0.136</b> $\pm 0.012$           | 0.139 $\pm 0.015$        | 0.137 $\pm 0.012$        | 0.140 $\pm 0.015$                     | 0.139 $\pm 0.014$        | 0.138 $\pm 0.014$        |
| K-L Divergence           | Electricity | 0.019 $\pm 0.030$                  | 0.020 $\pm 0.018$        | <b>0.008</b> $\pm 0.010$ | 0.033 $\pm 0.044$                     | 0.023 $\pm 0.021$        | 0.014 $\pm 0.010$        |
|                          | Solar       | 0.021 $\pm 0.013$                  | <b>0.012</b> $\pm 0.004$ | 0.013 $\pm 0.005$        | 0.015 $\pm 0.008$                     | 0.015 $\pm 0.009$        | 0.016 $\pm 0.012$        |
|                          | Wind        | 0.201 $\pm 0.040$                  | 0.186 $\pm 0.030$        | 0.202 $\pm 0.044$        | <b>0.180</b> $\pm 0.026$              | 0.182 $\pm 0.026$        | 0.203 $\pm 0.040$        |
|                          | Traffic     | 0.013 $\pm 0.007$                  | <b>0.009</b> $\pm 0.003$ | 0.011 $\pm 0.002$        | 0.010 $\pm 0.003$                     | <b>0.009</b> $\pm 0.002$ | 0.011 $\pm 0.005$        |
|                          | Taxi        | 0.006 $\pm 0.004$                  | <b>0.004</b> $\pm 0.003$ | 0.005 $\pm 0.003$        | 0.005 $\pm 0.002$                     | <b>0.004</b> $\pm 0.001$ | <b>0.004</b> $\pm 0.001$ |
|                          | Pedestrian  | <b>0.009</b> $\pm 0.004$           | 0.011 $\pm 0.005$        | <b>0.009</b> $\pm 0.004$ | 0.012 $\pm 0.004$                     | 0.012 $\pm 0.004$        | 0.010 $\pm 0.004$        |
|                          | Air         | 0.031 $\pm 0.012$                  | 0.025 $\pm 0.009$        | 0.026 $\pm 0.011$        | 0.028 $\pm 0.010$                     | 0.027 $\pm 0.009$        | <b>0.023</b> $\pm 0.008$ |
|                          | Temperature | <b>0.173</b> $\pm 0.027$           | 0.174 $\pm 0.039$        | 0.176 $\pm 0.027$        | 0.178 $\pm 0.036$                     | 0.181 $\pm 0.031$        | 0.186 $\pm 0.030$        |
|                          | Rain        | 0.014 $\pm 0.008$                  | <b>0.009</b> $\pm 0.004$ | 0.011 $\pm 0.003$        | 0.011 $\pm 0.004$                     | 0.015 $\pm 0.008$        | 0.012 $\pm 0.004$        |
|                          | NN5         | 0.048 $\pm 0.009$                  | 0.049 $\pm 0.018$        | <b>0.045</b> $\pm 0.007$ | 0.047 $\pm 0.012$                     | 0.054 $\pm 0.018$        | 0.053 $\pm 0.026$        |
|                          | Fred-MD     | 0.248 $\pm 0.109$                  | <b>0.197</b> $\pm 0.028$ | 0.201 $\pm 0.014$        | 0.201 $\pm 0.020$                     | <b>0.197</b> $\pm 0.020$ | 0.204 $\pm 0.024$        |
|                          | Exchange    | 1.629 $\pm 0.166$                  | 1.594 $\pm 0.151$        | 1.621 $\pm 0.126$        | 1.610 $\pm 0.159$                     | <b>1.590</b> $\pm 0.155$ | 1.635 $\pm 0.125$        |
| Count                    |             | 8                                  | 8                        | <b>9</b>                 | 4                                     | 9                        | 5                        |

## B. More Ablations

The ablation results for window aligning and domain prompting is shown in Table 4, indicating the necessity of these parts. Additionally, our prompting is different from TimeDP. TimeDP injects prompts via attention masking, while we use adaptive layer with multi-cond instead, avoiding interference with periodicity extraction in attention.

Table 4. Results for ablation study on different parts, including window aligning and domain prompting. The best results are **blod**.

| Ablation Type | Winformer         | W.O. window aligning | W.O. domain prompt | W.O. both         |
|---------------|-------------------|----------------------|--------------------|-------------------|
| Electricity   | 0.001 $\pm$ 0.002 | 0.002 $\pm$ 0.004    | 0.009 $\pm$ 0.006  | 0.009 $\pm$ 0.007 |
| <b>Solar</b>  | 0.035 $\pm$ 0.004 | 0.035 $\pm$ 0.003    | 0.402 $\pm$ 0.077  | 0.419 $\pm$ 0.073 |
| Wind          | 0.034 $\pm$ 0.014 | 0.034 $\pm$ 0.010    | 0.121 $\pm$ 0.088  | 0.115 $\pm$ 0.074 |
| Traffic       | 0.071 $\pm$ 0.005 | 0.073 $\pm$ 0.008    | 0.212 $\pm$ 0.037  | 0.221 $\pm$ 0.035 |
| Taxi          | 0.085 $\pm$ 0.010 | 0.088 $\pm$ 0.011    | 0.101 $\pm$ 0.031  | 0.127 $\pm$ 0.030 |
| Pedestrian    | 0.040 $\pm$ 0.008 | 0.041 $\pm$ 0.010    | 0.077 $\pm$ 0.012  | 0.057 $\pm$ 0.011 |
| Air           | 0.011 $\pm$ 0.002 | 0.012 $\pm$ 0.001    | 0.030 $\pm$ 0.032  | 0.031 $\pm$ 0.022 |
| temperature   | 0.230 $\pm$ 0.021 | 0.217 $\pm$ 0.026    | 0.60 $\pm$ 0.320   | 0.58 $\pm$ 0.330  |
| rain          | 0.036 $\pm$ 0.016 | 0.039 $\pm$ 0.024    | 0.192 $\pm$ 0.070  | 0.189 $\pm$ 0.058 |
| NN5           | 0.147 $\pm$ 0.008 | 0.152 $\pm$ 0.008    | 0.215 $\pm$ 0.023  | 0.259 $\pm$ 0.033 |
| Fred-MD       | 0.002 $\pm$ 0.001 | 0.002 $\pm$ 0.001    | 0.004 $\pm$ 0.001  | 0.004 $\pm$ 0.001 |
| Exchange      | 0.137 $\pm$ 0.012 | 0.139 $\pm$ 0.014    | 0.140 $\pm$ 0.044  | 0.138 $\pm$ 0.046 |

## C. Computation Costs

In this section, we try to analyze the computational costs of our method.

Theoretically, we assume the length of time-series is  $L_x$ , and the dimension is  $D$ . The computation complexity of the calculation for similarity scores, which is the dot-product of  $\mathbf{Q}$  and  $\mathbf{K}$ , is  $\mathcal{O}(L_x^2 D)$ . If the windows size is set to  $s$ , which means to compute with convolutional network with kernel size  $s$ , the computational complexity is  $\mathcal{O}(s^2)$ . We travel through all the time points of  $\mathbf{Q}$  and  $\mathbf{K}$ , the total computation of calculating  $\mathbf{S}$  becomes  $\mathcal{O}(s^2 \cdot L_x^2 D)$ , where the original similarity calculation cost is  $\mathcal{O}(L_x^2 D)$ .

We also tested the actual costs of Winformer as shown in Table 5. Notably, the cost time is for the whole pipeline, which is not limited to the ample attention. The training speed is to test how fast a denoising model can denoising a batch of data with gradient backpropagation during the training process. The sampling speed evaluates how fast a denoising model can make a prediction during the testing process. Although our window-wise alignment increases the complexity, in practical experiments, the actual computation consumption is acceptable with slight increase comparing with point-wise alignment because the calculation of convolutional neural operators has been optimized by machine learning frameworks.

Table 5. Computational costs for different kinds of alignment method. We evaluate the average speed of training and sampling of each batch (batch size=128).

| Model                                | Kernel Size | Train Speed (batch/s) | Sample Speed (batch/s) |
|--------------------------------------|-------------|-----------------------|------------------------|
| Point-wise Alignment                 | -           | 2.92                  | 10.57                  |
| Winformer<br>(Window-wise Alignment) | 7           | 2.58                  | 9.69                   |
|                                      | 13          | 2.33                  | 9.13                   |
|                                      | 25          | 2.30                  | 8.95                   |

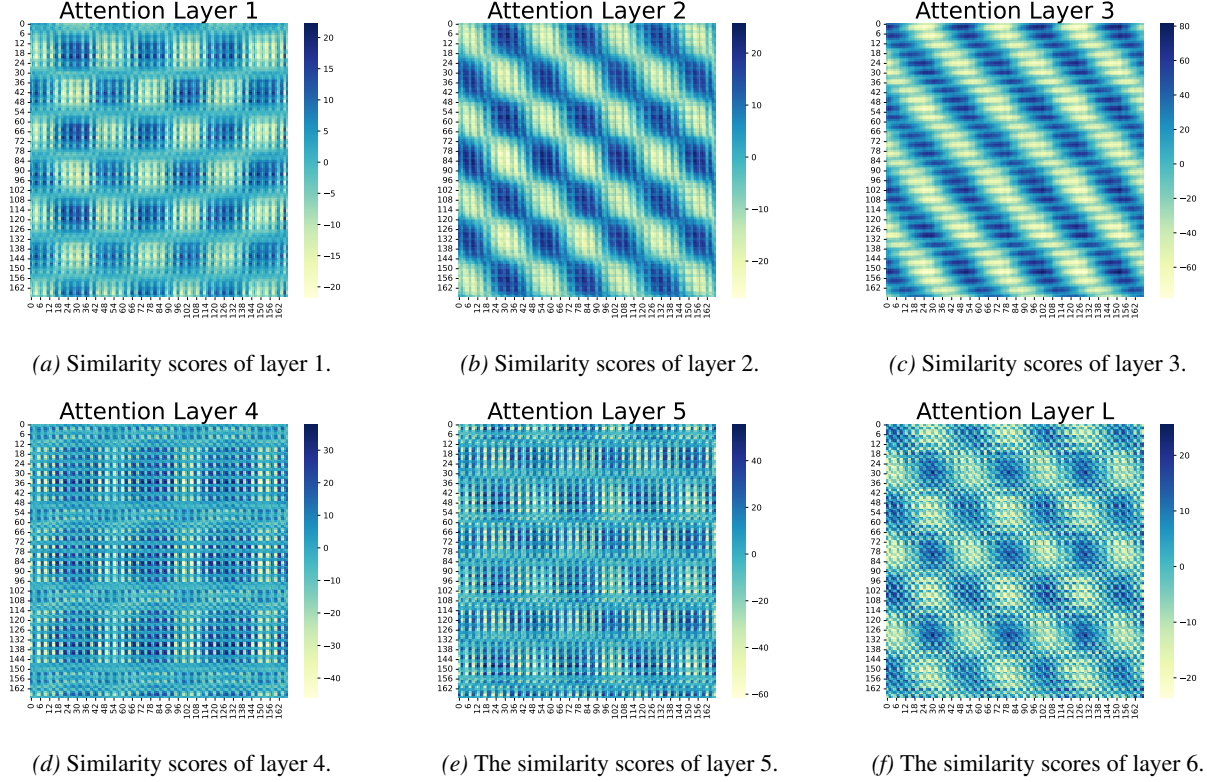


Figure 8. Visualization of the similarity matrices for synthetic data. On the synthetic data, the attention map for multiple layers indicates that periodicity is one of the primary characteristics reflected in the attention map.

## D. Visualizations and Discussions

We provide extended visualized figures on real-world datasets, including the visualization of the attention similarity matrix, the visualization of the denoising process and the visualization of the generated time-series. We further discuss about these visualized figures in the following subsections.

### D.1. Visualization of the Attention

In this subsection, we want to explore how can the Ample attention help the diffusion model in recovering the time-series patterns. We conduct experiments on the synthetic datasets to discover whether the model utilize the periodic pattern in denoising, which inspire us to enhance the periodic features with the Ample attention. We also verify the effectiveness on periodic enhancement by visualizing the similarity score before and after window-wise alignment.

#### D.1.1. VISUALIZATION ON SYNTHETIC DATA

We conduct the experiments on the synthetic datasets, which contains various periodic patterns as defined in Eq. 19. By observing the phenomena on the synthetic dataset, we can explore how the denoising model works with periodic capturing. We visualized the similarity score of the attention mechanism in Figure 8. In the heatmap of the score, the darker the color represents the deeper similarity of the time-series points. These visualized images show periodic cycles with repetitive square-like patterns. From layer 1 to Layer  $L$  ( $L = 6$ ), we can find that the patterns may shift with constant steps. The phenomena enlighten us that the periodic features are essential for transformer-based time-series denoising models.

#### D.1.2. VISUALIZATION ON REAL-WORLD DATA

We also visualized the similarity score matrices of the self-attention mechanism during the model’s denoising process on a portion of real-world data, as shown in Figure 9. This figure corresponds to the taxi dataset, which contains strong periodic patterns. The three subgraphs represent the kernel for window-wise alignment, the similarity score matrix before alignment,

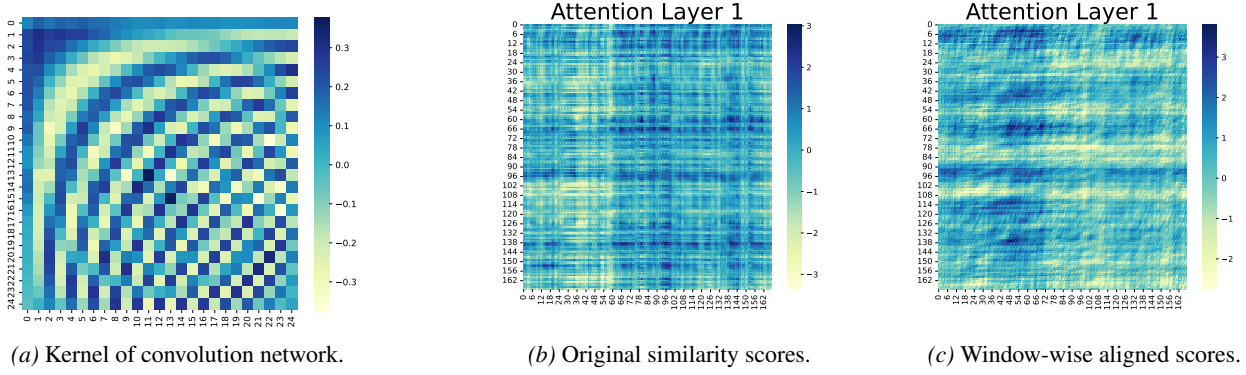


Figure 9. Visualization of the kernel for window-wise alignment, the similarity matrix before alignment, and the similarity matrix after window-wise alignment for dataset Taxi.

and the similarity matrix after window-wise alignment. The time-series data in the selected real-world datasets exhibit good periodicity, so their similarity matrices show a grid pattern. After window-wise alignment, we notice that the long-range grids become more distinct, indicating the window-wise alignment fuse features inside the observation window. Thus, our method can enhance the periodic features.

## D.2. Visualization for alignment comparison

To explore the differences in the effectiveness of the three kinds alignments, namely point-wise, patch-wise and window-wise, in extracting complex periodicity, we conducted result visualization on taxi dataset. Taxi dataset contains complex periodicity structure, as not only presents the daily cycle but also includes the detailed periodic changes of morning and evening peaks. We visualized the series generated with point-wise, patch-wise and window-wise alignment in Fig. 10. We can find that, window-wise method could capture the periodicity better than other methods, by well establishing periodicity structures.

## D.3. Visualization of the Generated Time-series

We visualized some time-series data generated by the Winformer, as shown in Fig. 11. The solar dataset contains a simple daily cycle. These generated series, containing obvious periodicity, indicates that the Winformer can effectively capture the periodic features. This confirms that our window-wise alignment can enhance the capturing of periodicity.

## D.4. Visualization of the Denoising Process

To further explore the denoising process in time-series generation tasks, we visualized the sequences during the denoising process on real-world dataset, as shown in Figure 12. The Solar datasets contains obvious daily cycles. By observing these images, we can find that in the denoising process of time-series data, long-range periodic information is captured first and displayed initially. Other detailed information will be revealed in later steps of denoising. Therefore, we propose the hypothesis that in the view of denoising model, the basic components for time-series data are frequency features instead of temporal points. That's explain and verify why the Winformer can achieve better performance against SOTA baselines.

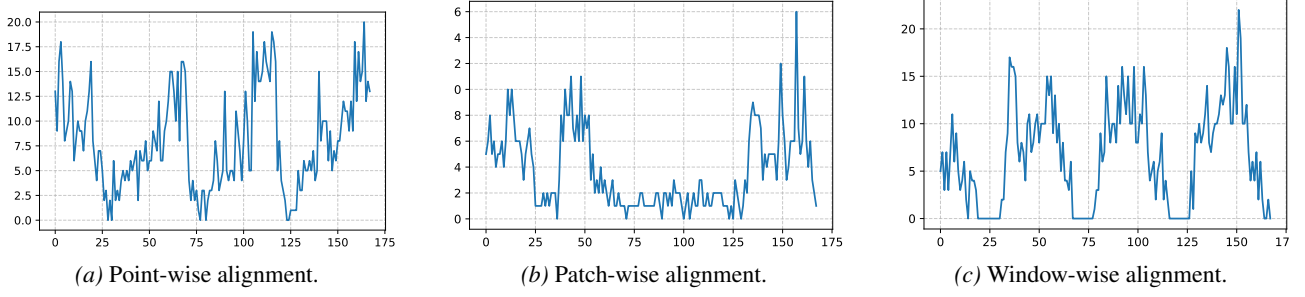


Figure 10. Visualization of the series generated by point-wise, patch-wise and window-wise alignment for dataset Taxi. This example demonstrates that window-wise generated samples capture periodicity more effectively.

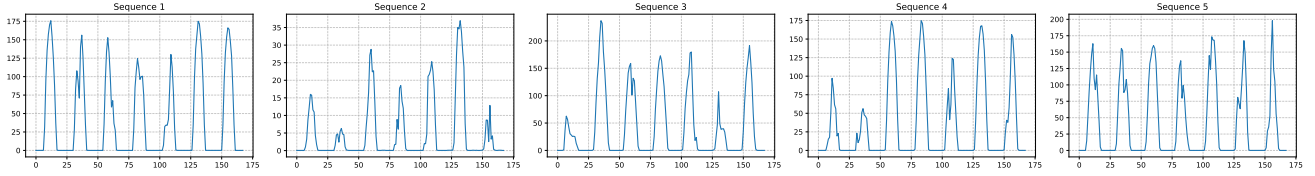


Figure 11. The generated time-series data for dataset Solar, containing obvious periodicity.

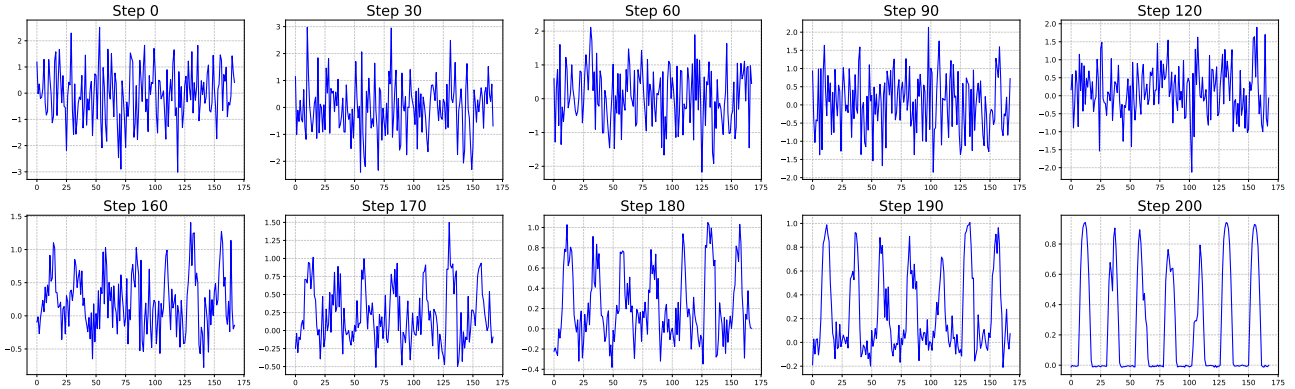


Figure 12. Denoising Process for the dataset of Solar, showing how periodicity reveals throughout the denoising process.

## E. Domain Discrepancy

We add a supplementary evaluation on domain discrepancy. Specifically, we evaluate the KL-divergence between different domains. The results are shown in Fig 13. To facilitate observation, all data in the figure are logarithmically transformed (log-transformed). For most datasets, the KL divergence between the generated data and the real data in their respective domains is the lowest, indicating that the domain-specific features are well learned.

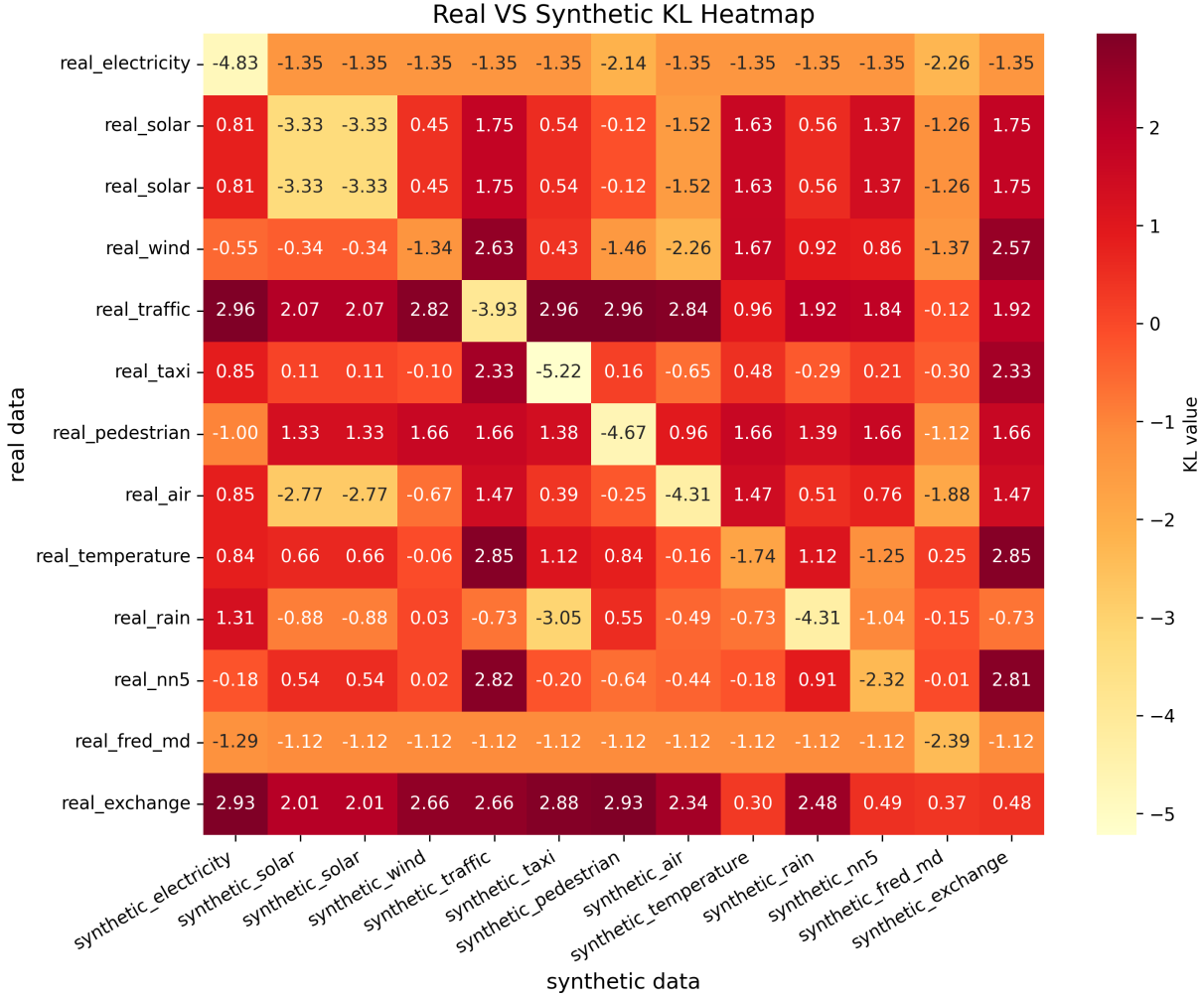


Figure 13. The domain discrepancy among datasets. For most datasets, domain discrepancy is evident, indicating that the domain-specific features are well learned.

## F. Expansion form

### F.1. Integrating with Trend decomposition.

We integrate Winformer with a basic trend adaptation (Autoformer-style) in Table 6, showing it has the potential to further improve performance on datasets with prominent trends.

### F.2. Multivariate Time-series Generation

Our model outperforms baseline on PEMS datasets, which are multivariate time-series datasets with complex cross-channel dependencies, with results in Table 7. We just inject the cross-domain dependencies (captured by a GCN) as condition to our model, and our model can realize the channel jointly generation with the dependencies.

Table 6. Integrating Winformer with Trend decomposition.

| Alignment Type | Winformer                             |                     | Winformer+Trend                       |                                       |
|----------------|---------------------------------------|---------------------|---------------------------------------|---------------------------------------|
| Metric         | MMD                                   | K-L                 | MMD                                   | K-L                                   |
| Wind           | $0.034_{\pm 0.014}$                   | $0.202_{\pm 0.044}$ | <b><math>0.030_{\pm 0.009}</math></b> | <b><math>0.200_{\pm 0.041}</math></b> |
| Fred-MD        | <b><math>0.002_{\pm 0.001}</math></b> | $0.201_{\pm 0.014}$ | <b><math>0.002_{\pm 0.001}</math></b> | <b><math>0.199_{\pm 0.017}</math></b> |
| Temperature    | $0.230_{\pm 0.021}$                   | $0.176_{\pm 0.027}$ | <b><math>0.202_{\pm 0.025}</math></b> | <b><math>0.170_{\pm 0.028}</math></b> |

Table 7. Results on PEMS datasets (cross-channel datasets).

| Model  | TimeDP |      | Winformer   |             |
|--------|--------|------|-------------|-------------|
| Metric | KL     | MMD  | KL          | MMD         |
| PEMS03 | 1.67   | 0.34 | <b>1.53</b> | <b>0.29</b> |
| PEMS04 | 2.29   | 0.33 | <b>1.88</b> | <b>0.21</b> |
| PEMS08 | 1.80   | 0.34 | <b>1.61</b> | <b>0.28</b> |

### F.3. Downstream Evaluation

Additional results show our practical value in Table 8 and 9, for classification and forecasting as you recommended. The predictor and classifier are vanilla transformer, and our method can help them to achieve better or comparable performance.

Table 8. Results on Application (multivariate forecasting).

| Model  | TSTR   |        | Original |        |
|--------|--------|--------|----------|--------|
| Metric | MSE    | MAE    | MSE      | MAE    |
| ETTh1  | 0.7411 | 0.6727 | 0.7682   | 0.6898 |
| ETTh2  | 1.3013 | 0.9295 | 1.3159   | 0.9352 |
| ETTm1  | 0.4974 | 0.5013 | 0.4997   | 0.502  |
| ETTm2  | 0.3881 | 0.4851 | 0.3785   | 0.4742 |
| ECL    | 0.2435 | 0.3432 | 0.2552   | 0.3481 |

### F.4. On Medical Datasets

To directly address the concern on irregular medical data, we now evaluate on three additional medical datasets. As shown in Table 10, Winformer consistently outperforms both TimeDP and DiT across all three datasets on both metrics.

Table 9. Results on Application (Classification).

| Model                | Add sythetic data | Original |
|----------------------|-------------------|----------|
| EthanolConcentration | 0.2776            | 0.2738   |
| FaceDetection        | 0.6827            | 0.6765   |
| Handwriting          | 0.3835            | 0.3753   |
| Heartbeat            | 0.7756            | 0.7707   |
| JapaneseVowels       | 0.9757            | 0.9784   |
| PEMS-SF              | 0.8382            | 0.8324   |
| SpokenArabicDigits   | 0.9859            | 0.9845   |
| UWaveGestureLibrary  | 0.8625            | 0.8594   |

Table 10. Evaluation on Medical Data.

|         | Winformer(Ours) |        | TimeDP |        | DiT   |        |
|---------|-----------------|--------|--------|--------|-------|--------|
| metric  | KL              | MMD    | KL     | MMD    | KL    | MMD    |
| PTB-XL  | 1.424           | 0.3496 | 1.552  | 0.3532 | 1.446 | 0.3512 |
| illness | 1.628           | 0.5979 | 1.6455 | 0.6708 | 1.637 | 0.6262 |
| Health  | 1.502           | 0.4174 | 1.685  | 0.4377 | 1.613 | 0.4283 |

## G. Experiment Details

We further report the detailed settings of the experiments, including information of the measurement, datasets and implementation.

### G.1. Metrics and Measurement

In this section, we describe the metrics and other settings which related to the measurement of the model’s performance. First, we formulate the key metrics evaluating the performance of the time-series generation. Then we describe the detail settings for repeated experiments and how we get the reported results with error bounds.

#### G.1.1. FORMULATED METRICS

In this paper, we adopt two metrics in measuring the quality of the generated time-series data. Firstly, we define a real time-series data with  $L$  length and  $D$  channels as  $\mathbf{X} = [\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_L] \in \mathbb{R}^{L \times D}$ , and the synthetic data is  $\hat{\mathbf{X}} = [\hat{\mathbf{X}}_1, \hat{\mathbf{X}}_2, \dots, \hat{\mathbf{X}}_L] \in \mathbb{R}^{L \times D}$ . Then we formulate the metrics of Maximum Mean Discrepancy and Kullback-Leibler Divergence as follow.

**(1) Maximum Mean Discrepancy (MMD).** The MMD is a distribution similarity evaluation method. Specifically, we transform the time-series data into a high-dimension space by  $\Phi(\cdot, \cdot)$ . Then we calculates the average of the results obtained by the kernel to get the MMD, which is formulated as follow:

$$L_{\text{MMD}} = \frac{\sum_{i=1}^N \Phi(\mathbf{X}_i, \mathbf{X}_i)}{N} + \frac{\sum_{i=1}^N \Phi(\hat{\mathbf{X}}_i, \hat{\mathbf{X}}_i)}{N} - 2 \frac{\sum_{i=1}^N \Phi(\mathbf{X}_i, \hat{\mathbf{X}}_i)}{N} , \quad (15)$$

where  $\Phi(\cdot, \cdot)$  is implemented by the radial basis function kernel.

**(2) Kullback-Leibler Divergence(K-L).** The K-L is a common metric measuring the similarity between real and synthetic data.

$$L_{\text{K-L}} = \sum_{i=1}^K P(\mathbf{X}) \log \left( \frac{P(\mathbf{X})}{Q(\hat{\mathbf{X}})} \right) , \quad (16)$$

where  $P(\cdot)$  and  $Q(\cdot)$  are mapping functions to obtain the distribution of the data by reforming the data into histogram, which includes  $K$  indexes in total.

#### G.1.2. REPEATED EXPERIMENTS

All experiments are repeated five times with random seeds ranging from 2021 to 2025. To demonstrate the comprehensive effect of the model, we report the average results with their standard deviation of the repeated experiments.

## G.2. Datasets

We conducted the time-series generation experiment on 12 real-world datasets and a synthetic dataset. The description of the datasets are as follow.

#### G.2.1. REAL-WORLD DATASETS

In this paper, we conduct the experiments following the setting of the TimeDP (Huang et al., 2025). The experiments contain 12 real-world datasets from four domains, including energy, economic, weather and transportation. The pre-processed datasets are open-sourced by TimeDP<sup>1</sup>. We list the details of the datasets in Table 11.

#### G.2.2. SYNTHETIC DATASET

The synthetic datasets contains time-series data which are the combination of the sinusoidal signal with different periods. Specifically, let  $\mathbf{X}_1(t)$  be the sinusoidal signal with a cycle of 50, which is defined as follow:

$$\mathbf{X}_1(t) = \sin\left(\frac{2\pi}{50}t\right) , \quad (17)$$

<sup>1</sup><https://huggingface.co/datasets/YukhoW/TimeDP-Data/blob/main/TimeDP-Data.zip>

Table 11. Details for the real-world datasets.

| Domain         | Dataset     | Variables | Sampling interval | Description                     | Source                 |
|----------------|-------------|-----------|-------------------|---------------------------------|------------------------|
| Energy         | Electricity | 321       | 1 hour            | the electricity consumption     | UCI                    |
|                | Solar       | 137       | 1 hour            | the solar power production      | State of Alabama       |
|                | Wind        | 1         | 4 second          | the wind power production       | AEMO                   |
| Weather        | Air         | 270       | 1 hour            | the air quality levels          | KDDCup2018             |
|                | Temperature | 422       | 1 day             | the temperature observations    | Australia              |
|                | Rain        | 422       | 1 day             | the rain forecast               | Australia              |
| Transportation | Traffic     | 963       | 1 hour            | the occupancy rate of car lanes | San Francisco bay      |
|                | Taxi        | 1214      | 30 minutes        | the taxi rides                  | New York               |
|                | Pedestrian  | 1         | 1 hour            | the pedestrian counts           | Melbourne city         |
| Economic       | NN5         | 111       | 1 day             | the cash withdrawals from ATMs  | UK                     |
|                | Fred-MD     | 107       | 1 month           | the macro-economic indicators   | Federal Reserve Bank   |
|                | Exchange    | 8         | 1 day             | the exchange rate               | Reports of 8 countries |

and  $\mathbf{X}_2(t)$  be the sinusoidal signal with a cycle of 5 defined as:

$$\mathbf{X}_2(t) = \sin\left(\frac{2\pi}{5}t\right) . \quad (18)$$

Then, we can obtain the synthetic data by superposing these two series as

$$\mathbf{X}(t) = \mathbf{X}_1(t) + \mathbf{X}_2(t) . \quad (19)$$

Thus, we get the synthetic data  $\mathbf{X}(t)$  which contains two types of cycles. We sample 20000 time steps and split the sequences into 168-length time series data. By conducting denoising method on the synthetic data, we can explore how the Ample attention and the Winformer helps in frequency capturing and cross-domain time-series generation. From visualization of the denoising process, we can find that the model firstly recovers the large-range periodic patterns. This motivates us to enhance the periodic pattern capturing by window-wise alignment, which is exactly our method does.

### G.3. Implementation Settings

In this section, we report the detailed implementation, including hyper-parameters and environments.

#### G.3.1. HYPER PARAMETERS

Our method, namely Winformer, is a transformer-based denoising model, consisting of several encoder blocks and a conditioning block. We have listed the detailed hyper-parameters of each component, which are shown in Table 12.

#### G.3.2. HARD-WARES AND ENVIRONMENTS

The experiments are conducted with an Ascend 910b GPU, with 32GB memory. Our model relies on public environment libraries, including Python, MindSpore, etc. The framework is based on the source code of TimeDP<sup>2</sup>. Our codes are reported in the supplementary material. More detailed environment and installation tips are reported in the ReadMe file in codes folder.

<sup>2</sup><https://github.com/microsoft/TimeCraft/tree/main/TimeDP>

Table 12. Hyper-parameters for the Winformer.

| Cateryory                 | Module               | Name                    | Value                           |
|---------------------------|----------------------|-------------------------|---------------------------------|
| Architecture of Winformer | Conditioning block   | Dimension               | 32                              |
|                           |                      | Channel                 | 1                               |
|                           |                      | Latent                  | 1                               |
|                           | Hybrid encoder block | Channel                 | 1                               |
|                           |                      | Hidden size             | 512                             |
|                           |                      | Vanilla attention heads | 8                               |
|                           |                      | Ample attention heads   | 8                               |
|                           |                      | MLP ratio               | 4.0                             |
|                           |                      | Kernel type             | Conv2d                          |
|                           |                      | Kernel initialization   | discrete cosine transform (DCT) |
|                           |                      | Kernel size             | 25                              |
|                           | DiT encoder block    | Hybrid encoder block    | 6                               |
|                           |                      | Channel                 | 1                               |
|                           |                      | Vanilla attention heads | 16                              |
|                           |                      | MLP ratio               | 4.0                             |
| Forward/Reverse Process   |                      | Noising/Denoising steps | 200                             |
|                           |                      | Loss type               | L1 loss                         |
| Trainer                   |                      | Batch size              | 128                             |
|                           |                      | Learning rate           | 5e-5                            |
|                           |                      | Train steps             | 50,000                          |

---

## H. Task Introduction

### H.1. Generation VS. Forecasting

Forecasting tasks emphasize on detailed perception for instances, while generation tasks requires global structure awareness for distributions. Forecasting and imputation tasks focus on estimating the series to the groundtruth under know observations in every single instances, but generation is to estimate the global distribution to the whole dataset. Generation tasks impose stronger requirements on more widely existing structures while downplaying the unique characteristics of individual samples. That’s why we select the generation tasks to explore how could the model learn the periodicity structure better.

### H.2. Observations for Cross-domain Generation

Cross-domain generation requires models to learn and transfer periodic patterns across diverse domains. As illustrated in Fig. 14(a), series from domain 1 and domain 2 show different periodicity, which is challenging for existing point-wise and patch-wise methods. Capturing the domain-specific periodicity directly tests the architecture’s ability to generalize periodicity.

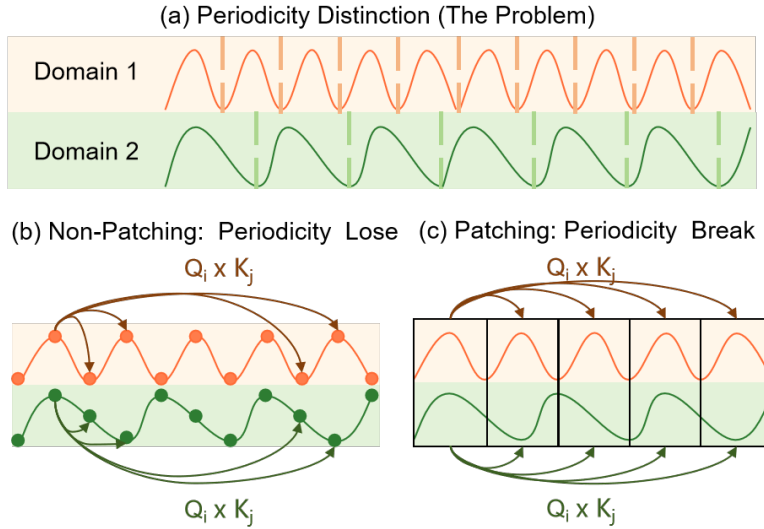


Figure 14. The observation for cross-domain time-series generation. For non-patching (point-wise) methods, the model understands each point relatively independently and lacks the ability to capture the correlation between adjacent time steps, which is detrimental to the acquisition of pattern structures. For patching methods, with information isolated and compressed at equal intervals, fixed windows will disrupt the periods of some sequences.

### H.3. The Discovery of the Adaptive Window-Wise Architecture

The design of Winformer’s adaptive window-wise architecture stems from a key observation, that is the limitations of existing point-wise and patch-wise paradigms. Traditional point-wise attention struggles with complex dependencies and fails to model cyclic structures, as it only captures point-to-point interactions. As illustrated in Fig. 14(b), it’s difficult to identify repetitive structures when time-series are numerically sensitive in continues space. Patch-wise methods fragment time-series into fixed segments, breaking evolving periodic patterns and requiring additional distribution restoration steps. Especially for cross-domain generation, models may process time-series with different windows, patching with a fixed segments could not be suitable for every domain, as illustrated in Fig. 14(c). These flaws motivated us to rethink the “processing unit” of attention: Could we shift the discrete points or patches to continuous windows that naturally encapsulate periodic cycles? Compared to the point-wise methods, the window-wise methods can summary local features and simplified attention’s work. Compared to the patch-wise mwthods, the window alignment is sliding, which allows to adapt different domains, as for every domain there is expected to have partial of the windows can contain the proper periodicity features. With these observations, we start for theoretical analysis to explore how to capture the periodicity with proper approaches.

## I. Theoretical Explanation and Proof

In this section, we supplement theoretical backgrounds to facilitate presentation of the motivation and the design concepts. With these further explanation, we can better understand the underlying reasons and deduction process.

### I.1. The Fourier Transform

The Fourier Transform (Duhamel & Vetterli, 1990) is a mathematical operator that converts signal  $\mathbf{x}$  to its frequency domain representation  $\tilde{\mathbf{x}}$ . It is defined as a function  $\mathcal{F}$  as:

$$\mathcal{F}\{u\}(\xi) = \int_{-\infty}^{+\infty} u(x)e^{-2\pi jx\xi} dx \quad . \quad (20)$$

The  $u(x)$  is the input function to signal  $\mathbf{x}$ 's formulation. Then, we set the integral results as individual spectral components for different frequency  $\xi$ .

### I.2. Calculating Similarity Score in Frequency Domain

The attention mechanism, which calculates as follow:

$$\mathcal{A} = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}}\right)\mathbf{V} \quad , \quad (21)$$

where  $\mathbf{Q}$  and  $\mathbf{K}$  are mapped features from input  $x$ . To calculate the similarity score  $\mathbf{S} = \mathbf{Q}\mathbf{K}^T$  in the frequency domain, we can directly transform the input data with Fourier transform and calculate the score using the Fourier components. However, such a process is time-consuming and inflexible and with Fourier operators. For a learning-based model, we prefer a learning surrogator for this operation, which can be adaptively parameterized cooperating with the model. Thus, we need to further explore the simplified calculation of the score  $\mathbf{S}$ .

### I.3. Convolution Theorem

To further explore the simplified calculation of the score, we have to consider how to transform the dot-product operator in frequency domain. As the convolution theorem (McGillem & Cooper, 1991) states that:

$$\mathcal{F}\{u * v\} = \mathcal{F}\{u\} \cdot \mathcal{F}\{v\} \quad , \quad (22)$$

where  $*$  is the convolution and the operator  $\cdot$  represents the point-wise multiplication. The above derivations also apply for the discrete Fourier transform (DFT). Thus, there is the possibility of transforming the dot-production in the frequency domain into the convolution in the temporal domain. As a result, we can directly calculate the score  $\mathbf{S}$  with convolution network, in the place of a series of the calculation for frequency transformation.

### I.4. Parseval's Theorem

For discrete time signals, we consider a time-series data  $x$  with length  $n$ . The Parseval's Theorem states:

$$\sum_{n=0}^{N-1} |x[n]|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X[k]|^2, \quad (23)$$

where  $X[k]$  is the discrete Fourier transform (DFT) of  $x[n]$ . The Parseval's Theorem enables us to transfer correlation of frequency domain into that to time domain. Thus, we can adapt the deduction of Eq. 11 in the attention mechanism, leading to the final format of the Ample attention shown in Eq. 12.

### I.5. Relationship Between DCT and DFT Relationship

**Lemma I.1.** *Let  $x[n] \in \mathbb{R}$  be a 1D real-valued finite-length signal of length  $N$  ( $n = 0, 1, \dots, N - 1$ ). The  $N$ -point Type II Discrete Cosine Transform (DCT-II) of  $x[n]$  is a **special form** of the Discrete Fourier Transform (DFT), which can be exactly derived from the DFT of the even-symmetric extended version of  $x[n]$ .*

*Proof.* Firstly, we define the notations for DCT and DFT, as well as the periodic even-symmetric extension of the series  $x[n]$ :

- $N$ -point 1D DFT:  $X[k] = \text{1D-DFT}\{x[n]\} = \sum_{n=0}^{N-1} x[n]e^{-j\frac{2\pi}{N}kn}$ ,  $k = 0, 1, \dots, N-1$ ,  $j = \sqrt{-1}$ ;
- $N$ -point 1D DCT-II:  $X_C[k] = \text{1D-DCT-II}\{x[n]\} = \sum_{n=0}^{N-1} x[n] \cos\left(\frac{\pi}{N}\left(k + \frac{1}{2}\right)n\right)$ ,  $k = 0, 1, \dots, N-1$ ;
- $4N$ -point periodic even-symmetric extension of  $x[n]$ :  $x_{\text{sym}}[n]$ , satisfying  $x_{\text{sym}}[n] = x_{\text{sym}}[4N - n]$  for all  $n$ .

By  $\cos \theta = \frac{e^{j\theta} + e^{-j\theta}}{2}$ , the cosine kernel is rewritten as a combination of complex exponentials, which is the core kernel of DFT:

$$\cos\left(\frac{\pi}{N}\left(k + \frac{1}{2}\right)n\right) = \frac{1}{2} \left[ e^{j\frac{\pi(2k+1)n}{2N}} + e^{-j\frac{\pi(2k+1)n}{2N}} \right]. \quad (24)$$

Substituting into the DCT-II definition gives:

$$X_C[k] = \frac{1}{2} \sum_{n=0}^{N-1} x[n] \left[ e^{j\frac{\pi(2k+1)n}{2N}} + e^{-j\frac{\pi(2k+1)n}{2N}} \right]. \quad (25)$$

Then, the  $4N$ -point even-symmetric extension of the signal is defined as:

$$x_{\text{sym}}[n] = \begin{cases} x[n], & 0 \leq n \leq N-1, \\ x[2N-1-n], & N \leq n \leq 2N-1, \\ 0, & 2N \leq n \leq 4N-1. \end{cases} \quad (26)$$

The DFT of  $x_{\text{sym}}[n]$  is  $X_{\text{sym}}[m] = \sum_{n=0}^{2N-1} x_{\text{sym}}[n]e^{-j\frac{\pi mn}{2N}}$  for  $m = 0, 1, \dots, 4N-1$ . Splitting the summation and substituting  $n' = 2N-1-n$  for the second term yields:

$$X_{\text{sym}}[m] = \sum_{n=0}^{N-1} x[n] \left[ e^{-j\frac{\pi mn}{2N}} + e^{-j\frac{\pi m(2N-1-n)}{2N}} e^{j\frac{\pi mn}{2N}} \right]. \quad (27)$$

For  $m = 2k+1$ , simplify the exponential term  $e^{-j\frac{\pi(2k+1)(2N-1-n)}{2N}} = e^{-j\pi(2k+1)}e^{j\frac{\pi(2k+1)n}{2N}} = -e^{j\frac{\pi(2k+1)n}{2N}}$ , considering  $e^{-j\pi(2k+1)} = -1$ . Substitute into Eq.(27):

$$X_{\text{sym}}[2k+1] = - \sum_{n=0}^{N-1} x[n] \left[ e^{j\frac{\pi(2k+1)n}{2N}} + e^{-j\frac{\pi(2k+1)n}{2N}} \right]. \quad (28)$$

Comparing Eq.(25) and Eq.(28), the summation in Eq.(28) is exactly  $2X_C[k]$ . Rearranging gives the key equivalence:

$$X_C[k] = -\frac{1}{2} \cdot \text{DFT}\{x_{\text{sym}}[n]\} \Big|_{m=2k+1}. \quad (29)$$

This formula shows the DCT-II is uniquely determined by the DFT of the even-symmetric extended signal. No approximations or ad-hoc rules are involved in the derivation. Thus the theorem holds.  $\square$